DISPENSARY MANAGEMENT SYSTEM

Software Requirements Specification

Release 1.0 - patient records, consultations, prescriptions, dispensing and queue management

Document	01-Software-Requirements-Specification.pdf
Version	1.0.0
Prepared	12 October 2026
Status	Draft for owner review - implementation baseline for release 1.0
Purpose	Defines testable functional and non-functional requirements, business rules and acceptance criteria.

Document set

File	Content
01-Software-Requirements-Specification.pdf	What the system must do: scope, roles, FR/NFR, business rules, workflows, use cases, validation, security, audit, open questions, traceability.
02-Database-and-System-Architecture.pdf	How it is built: architecture, components, sequences, data model, data dictionary, DDL, concurrency, deployment.
03-REST-API-Documentation.pdf	Every endpoint with roles, parameters, examples, validation, errors and transactions (generated from openapi.yaml).
openapi.yaml	Machine-readable OpenAPI 3.1 contract - the source of truth for the API.
backend-requirements.md / frontend-requirements.md	Implementation instructions for Claude Code.

All documents in the set are generated from the same sources (database migration, openapi.yaml and requirement catalogues) and are cross-checked automatically; identifiers such as FR-012, BR-Q-03 or ERR codes are stable across documents.

Contents

1. Introduction **4**
1.1 Purpose 4
1.2 Intended audience 4
1.3 Conventions 4
1.4 Glossary 4
1.5 References 5

2. Project overview **6**
2.1 Overview 6
2.2 Problem statement 6
2.3 Objectives 6
2.4 Scope (release 1.0) 6
2.5 Out of scope (release 1.0) 7
2.6 Technology stack 7

3. Requirements analysis **8**
3.1 Ambiguities and decisions 8
3.2 The doctor's queue button - analysis 10

4. User roles and permissions **11**
4.1 Roles 11
4.2 Capabilities by role (derived from openapi.yaml x-roles) 11
4.3 Endpoint permission matrix 12

5. Functional requirements **15**
5.1 Summary 15
5.2 Detailed requirements and acceptance criteria 16

6. Non-functional requirements **28**
6.1 Non-functional requirements table 28
6.2 How performance targets are tested 30

7. Business rules **31**
7.1 General 31
7.2 Patients 31
7.3 Consultations 31
7.4 Prescriptions 31
7.5 Dispensing 32
7.6 Queue 32
7.7 Bookings 33
7.8 Status lifecycles (canonical) 33

8. End-to-end workflows **38**
8.1 Workflow A - Existing patient consultation 38
8.2 Workflow B - New patient registration and consultation 38
8.3 Workflow C - Telephone booking 39
8.4 Workflow D - Public queue display 39
8.5 Workflow E - Calling the next patient 39

9. Use cases **40**

10. Input validation rules **42**

11. Error-handling requirements **45**
11.1 Error code catalogue 45

12. Security and privacy requirements **49**

12.1 Authentication and sessions 49
12.2 Authorisation and least privilege 49
12.3 Application security 49
12.4 Privacy and clinical data 49
12.5 Backup, retention and deletion 50
12.6 Legal, clinical and operational items the owner must confirm before production 50

13. Audit logging requirements 51
13.1 Audit action catalogue 51

14. Data and reporting 54

15. Assumptions, constraints and unresolved questions 55
15.1 Assumptions 55
15.2 Constraints 55
15.3 Unresolved questions (with proposed defaults used in this release) 55

16. Future enhancements 57

17. Traceability matrix 58

Appendix A. Test catalogue 61
A.1 Backend tests 61
A.2 Frontend tests 64

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines what release 1.0 of the Dispensary Management System (DMS) must do. It is the requirements baseline for the companion documents *02-Database-and-System-Architecture.pdf* (how the system is built), *03-REST-API-Documentation.pdf* / `openapi.yaml` (the API contract) and the implementation guides `backend-requirements.md` and `frontend-requirements.md`, which are handed to Claude Code for implementation.

1.2 Intended audience

- The dispensary owner and clinical lead, who must confirm the business decisions in section 15.
- Developers (human or Claude Code) implementing the backend and frontend.
- Testers deriving test cases from the acceptance criteria.

1.3 Conventions

- **MUST** requirements are release blockers; **SHOULD** requirements are planned for release 1.0 but may slip with the owner's approval.
- Identifiers are stable across all documents: `FR-nnn` functional, `NFR-nnn` non-functional, `BR-<area>-nn` business rule, `UC-nn` use case, `SEC-nn` security, `AUD-nn` audit, `EH-nn` error handling, `A-nn` assumption, `C-nn` constraint, `Q-nn` open question, `BT-nnn` / `FT-nnn` backend / frontend tests, `SCR-nn` frontend screens.
- Values marked **Proposed target** are the analyst's recommendation; they are measurable and testable but must be confirmed by the owner.
- `code style` denotes API fields, enum values, database columns or routes exactly as implemented.

1.4 Glossary

Term	Definition
Patient number	Business identifier of a patient (e.g. `P-0001`), printed on the clinic card, entered by staff. Distinct from the internal UUID `id`.
Consultation	One doctor-patient encounter: complaint, notes, 1-10 diagnoses and at most one active prescription. Immutable once saved.
Medical history	The ordered set of a patient's consultations with their diagnoses, prescriptions and dispensing results. Never a single overwritable field.
Prescription	The medicine order of a consultation (1-30 items). Lifecycle `PENDING -> IN_PROGRESS -> DISPENSED`, or `CANCELLED`.
Dispensing	The single record of what a pharmacist actually handed over, per item, with pharmacist and timestamp.
Queue session	A numbered queue for one clinic on one date (e.g. "Morning OPD - Room 1"). Numbers start at 1 in every session.
Queue entry / booking	A position in a queue session. "Booking" is the act of creating a queue entry by telephone or for a walk-in.
Now serving / current serving number	The number most recently called in a session (`currentServingNumber`).

Term	Definition
Call next	Doctor action that advances the queue: completes the current patient (optional) and calls the lowest waiting number.
Public queue display	Unauthenticated page showing only queue numbers, counts and status.
Idempotency key	Client-generated unique key sent with create requests so that retries never create duplicates.
Staff	Any authenticated user (ADMIN, DOCTOR, PHARMACIST). Patients are not authenticated users in release 1.0.

1.5 References

- `openapi.yaml` - OpenAPI 3.1 contract (49 operations, 79 schemas, 55 error codes).
- `02-Database-and-System-Architecture.pdf` - sections 3 (data model), 4 (data dictionary), 7 (transactions and concurrency).
- RFC 9457 Problem Details for HTTP APIs; OWASP ASVS (used as a checklist, not a compliance claim); WCAG 2.2.

2. Project overview

2.1 Overview

The DMS is a web application for a single dispensary (outpatient clinic with an in-house pharmacy). Doctors look up or register patients, review their medical history and record consultations with prescriptions. Prescriptions flow automatically to the pharmacists, who review and dispense them; doctors see the dispensing result. Staff register telephone and walk-in bookings and give each patient a queue number; doctors call patients in order and a public page shows which number is being served - without exposing any personal data.

The system is a modular monolith: a Next.js (TypeScript) frontend, a Spring Boot REST API and a PostgreSQL database, run with Docker Compose.

2.2 Problem statement

Paper registers and verbal hand-overs cause four recurring problems:

1. Previous consultations and prescriptions are hard to find, so doctors decide without the history.
2. Prescriptions travel to the pharmacy on paper; they get lost or misread, and the doctor cannot tell whether medicines were given or which were unavailable.
3. Queue numbers are issued manually; numbers get duplicated or skipped, and waiting patients repeatedly ask staff where the queue is.
4. There is no record of who viewed or changed a patient's record.

2.3 Objectives

ID	Objective	Measure (proposed target)
OBJ-1	Complete history at the point of care	100% of consultations recorded in the DMS are visible in the patient history within 1 s of saving (FR-016).
OBJ-2	Paperless consultation-to-pharmacy hand-over	Prescription visible to pharmacists within one polling interval (10 s) of submission (FR-021).
OBJ-3	Traceable dispensing	Every dispensed prescription records per-item quantity, pharmacist and timestamp (FR-028); zero duplicate dispensing records (FR-029).
OBJ-4	Reliable queue	Zero duplicate or skipped queue numbers under concurrency tests (FR-036, FR-039); public display updated within 7 s (NFR-004).
OBJ-5	Privacy by design	No personal data on public pages (FR-044); all access to patient records audited (FR-046).

2.4 Scope (release 1.0)

- Staff authentication, role-based authorisation (ADMIN, DOCTOR, PHARMACIST) and staff account administration.
- Patient registration, lookup by patient number, search, demographic updates, unknown date of birth handling.
- Medical history: conditions summary and paginated consultations with diagnoses, prescriptions and dispensing results.
- Consultations with diagnoses, clinical notes and a multi-item prescription saved atomically; prescription correction (cancel + re-issue).

- Pharmacist prescription queue, prescription review, start, dispense (full or partial with per-item record) and cancel.
- Lightweight medicine catalogue (names, strengths, forms, active flag) - **no stock or inventory**.
- Queue sessions, telephone and walk-in bookings, call next, call specific, complete, skip, requeue, cancel, open/close sessions.
- Public queue list and display with automatic updates (polling) and a private "my position" helper.
- Audit trail and admin audit viewer; data retention jobs; health checks; OpenAPI documentation.

2.5 Out of scope (release 1.0)

Each item is deliberately excluded; section 16 lists them as future enhancements.

- Medicine inventory, stock levels, batch/expiry tracking, purchasing, automatic "unavailable" detection.
- Patient accounts or login, online self-booking, SMS/e-mail/WhatsApp notifications.
- Billing, payments, insurance.
- Automated diagnosis, dose calculation, drug-interaction or allergy checking, medicine recommendations of any kind (BR-G-03).
- Printing of prescriptions, labels or queue tickets (browser print of screens is possible but not designed).
- Laboratory results, imaging, vitals, referrals, appointments with fixed time slots.
- Multi-branch operation, offline mode, mobile native apps, integration with national health systems.

2.6 Technology stack

The requested stack is adopted without change. Versions current at the preparation date are recommended (details in the implementation guides):

Layer	Choice
Frontend	Next.js 16 (App Router) + React 19 + TypeScript, Tailwind CSS 4, shadcn/ui, React Hook Form, Zod, TanStack Query 5
Backend	Java 21 LTS, Spring Boot 4.1.x (Spring Framework 7, Spring Security 7, Hibernate 7), Bean Validation, Maven, springdoc-openapi 3
Database	PostgreSQL 16, Flyway migrations
Tooling	Docker / Docker Compose, JUnit 5 + Mockito + Testcontainers, Vitest + React Testing Library + Playwright + axe-core
Real-time	HTTP polling with ETag (5 s) - justified in 02-Database-and-System-Architecture.pdf section 2.6

3. Requirements analysis

This section records the result of Phase 1 (analysis): contradictions, ambiguities and the default chosen for each. Defaults are used throughout the document set; items marked *Confirm* are repeated as open questions in section 15.3.

3.1 Ambiguities and decisions

ID	Source text / issue	Decision (proposed default)	Confirm
D-01	"Increase the patient queue number using a dedicated button" could mean create a booking, increase a waiting count, advance the serving number or call the next patient.	The button **calls the next patient**: it atomically completes the current patient (optional) and moves the lowest waiting number to SERVING, advancing the public "now serving" number. It never creates a booking or changes counts directly. Booking creation is a separate action (FR-036). See 3.2.	Q-03
D-02	"Three primary roles" include patients, but patients are public users; nobody is named to create staff accounts.	Authenticated roles are DOCTOR and PHARMACIST plus a minimal **ADMIN** role for account administration and audit review only (no clinical access). Patients are unauthenticated public queue users.	Q-12
D-03	"Patient ID or another agreed unique identifier" - source unknown.	`patientNumber`: staff-entered clinic-card number, 3-20 chars `[A-Z0-9-]`, normalised to upper case, unique, immutable. Internal key is a UUID. National ID is not collected.	Q-01, Q-13
D-04	"Age or date of birth" and "patients whose DOB is unknown".	Store `date_of_birth` always; when unknown, staff enter an estimated age and the system derives the DOB (today - N years) with `date_of_birth_estimated = true`. Age is always computed.	Q-19
D-05	"Cancelled, if permitted".	Cancellation permitted from PENDING/IN_PROGRESS by the prescribing doctor or any pharmacist, with a mandatory reason. DISPENSED and CANCELLED are terminal.	-
D-06	"Record what the doctor prescribed and what the pharmacist actually dispensed" vs. no inventory.	Prescribed items are immutable; the dispensing records per-item quantity (0..prescribed) with a note when short. "Unavailable medicine" means the pharmacist records it as not or partially dispensed, or the catalogue entry is deactivated so it cannot be prescribed.	Q-05
D-07	"Handle multiple doctors or service counters if supported".	Supported through multiple queue sessions per day (one per room/doctor, recommended). A shared session may also be served by several doctors; each call is atomic and the public page lists all numbers being served.	Q-15
D-08	"Reset or start a new queue session according to a clearly defined rule".	Numbering restarts at 1 for every queue session; sessions belong to one date, so numbering restarts at least daily. Past sessions are auto-closed nightly.	-
D-09	Telephone booking: same day or advance?	Bookings may be made for sessions dated today up to 30 days ahead (session must exist and be SCHEDULED or OPEN).	Q-07
D-10	Pharmacist needs "doctor and consultation details".	Pharmacists see prescribing doctor, consultation date/time and submission time, plus minimal patient data (name, number, sex, age, allergies) - not diagnoses or clinical notes (least privilege).	Q-02
D-11	Who registers bookings: "dispensary or pharmacist".	DOCTOR and PHARMACIST can create and cancel bookings and manage sessions; only DOCTOR can call/complete/skip patients.	Q-03
D-12	Workflow B partial failure (patient created, prescription fails).	Registration and consultation are separate transactions by design. A registered patient is a valid record on its own; if the consultation fails nothing of it is saved, the form keeps its data, and the retry uses the same idempotency key (BR-C-05, EH-13).	-
D-13	Real-time updates "or near real time".	Polling every 5 s with ETag/304 for queue views; 10 s for prescription lists. SSE is a documented future upgrade.	-

ID	Source text / issue	Decision (proposed default)	Confirm
D-14	"Regulation" not specified.	No compliance claim is made. Applicable law (e.g. data-protection legislation where the dispensary operates) must be confirmed by the owner (section 12.6).	Q-16

3.2 The doctor's queue button - analysis

The four candidate meanings are different operations and are deliberately not interchangeable:

Candidate meaning	Effect	Risk if the button did this	Release 1.0 design
Create a booking	New queue entry with the next number	A doctor's click would issue numbers to nobody and create phantom patients	Separate action "New booking" (FR-036), available to doctors and pharmacists
Increase a waiting count	Counter +1 without an entry	Counts no longer match real entries	Not offered - counts are always derived from entries
Advance the serving number	"Now serving" + 1 regardless of entries	Skips cancelled numbers or calls numbers that do not exist; double click skips a patient	Not offered as a raw counter
Call the next patient	Lowest WAITING entry → SERVING; now-serving number = that entry	None if done atomically with compare-and-set	**The button.** `POST /queue-sessions/{id}/call-next` (FR-039)

The button sends the serving number the doctor currently sees (`expectedLastCalledNumber`). The server advances only if that number is still current; otherwise it returns `409 QUEUE_STATE_CHANGED` and the screen refreshes. A double click, a network retry or two doctors pressing simultaneously therefore advance the queue exactly once - no patient is skipped silently. The button is labelled **"Complete & call next"** when a patient is being served (it completes that patient in the same transaction) and **"Call next"** otherwise. Separate **Skip (not present)** and **Complete** buttons exist for the current patient.

4. User roles and permissions

4.1 Roles

Role	Who	Can	Cannot
DOCTOR	Registered medical practitioner	Search/register/update patients, view full medical history, record consultations and prescriptions, cancel/re-issue own prescriptions, view dispensing status, manage queue sessions and bookings, call/complete/skip/requeue patients, view medicine catalogue	Dispense, manage users, view audit log
PHARMACIST	Pharmacist / dispenser	View the prescription queue and prescription details (minimal patient data), start, dispense and cancel prescriptions, manage medicine catalogue, manage queue sessions and bookings, requeue late patients	View diagnoses, clinical notes or full histories; register patients; call/ complete/skip patients; manage users
ADMIN	Owner or IT administrator	Manage staff accounts and roles, reset passwords, view audit log, manage medicine catalogue, list staff	Any clinical data (patients, consultations, prescriptions) unless the same person also holds DOCTOR or PHARMACIST
Public queue user	Patients and visitors (unauthenticated)	View today's queues and the public status of one queue	Anything else

A user may hold several roles (e.g. a doctor who also dispenses); permissions are the union. Every permission is enforced on the server; hiding a button in the UI is never the control (FR-006, SEC-05).

4.2 Capabilities by role (derived from `openapi.yaml` x-roles)

ADMIN (13 operations): Get the authenticated user; Change own password; List staff accounts; Create a staff account; Get a staff account; Update a staff account (name, roles, enabled, registration number); Set a temporary password for a user; Minimal directory of enabled staff (e.g. doctors for queue-session assignment); Search the medicine catalogue (prescription autocomplete and catalogue management); Add a medicine to the catalogue; Get a catalogue medicine; Update or (de)activate a catalogue medicine; Search the audit trail.

DOCTOR (31 operations): Get the authenticated user; Change own password; Minimal directory of enabled staff (e.g. doctors for queue-session assignment); Search patients by name fragment, phone number or patient-number prefix; Register a new patient; Look up a patient by patient number (doctor's patient-ID search); Get patient demographics; Update patient demographics and allergies; Get a patient's medical history (conditions summary + paginated consultations); Create a consultation with diagnoses and an optional prescription (one transaction); Get a consultation with diagnoses, prescriptions and dispensing status; Add (or re-issue) a prescription for an existing consultation; List prescriptions (pharmacist queue / doctor's own prescriptions); Get a prescription with items, minimal patient data and dispensing result; Cancel a prescription that has not been dispensed; Search the medicine catalogue (prescription autocomplete and catalogue management); Get a catalogue medicine; List queue sessions for a date; Create a queue session (e.g. today's clinic for a room/doctor); Get queue session status for staff (counts, now serving, next number); Open a scheduled session so patients can be called; Close a session (end of clinic); List bookings / queue entries of a session (staff only); Register a telephone or walk-in booking and assign the next queue number; Call the next waiting patient (the doctor's "Next patient" button); Get one booking /

queue entry; Call a specific waiting entry out of order (e.g. emergency or priority patient); Mark the served entry as completed without calling the next one; Mark a called patient as not present (no-show); Return a skipped patient to the waiting list (arrived late); Cancel a waiting booking (patient called to cancel).

PHARMACIST (22 operations): Get the authenticated user; Change own password; Minimal directory of enabled staff (e.g. doctors for queue-session assignment); List prescriptions (pharmacist queue / doctor's own prescriptions); Get a prescription with items, minimal patient data and dispensing result; Mark a prescription IN_PROGRESS (pharmacist has started preparing it); Confirm dispensing ("Mark as Dispensed") with per-item quantities; Cancel a prescription that has not been dispensed; Search the medicine catalogue (prescription autocomplete and catalogue management); Add a medicine to the catalogue; Get a catalogue medicine; Update or (de)activate a catalogue medicine; List queue sessions for a date; Create a queue session (e.g. today's clinic for a room/doctor); Get queue session status for staff (counts, now serving, next number); Open a scheduled session so patients can be called; Close a session (end of clinic); List bookings / queue entries of a session (staff only); Register a telephone or walk-in booking and assign the next queue number; Get one booking / queue entry; Return a skipped patient to the waiting list (arrived late); Cancel a waiting booking (patient called to cancel).

4.3 Endpoint permission matrix

"yes" = allowed, "-" = 403 FORBIDDEN, "public" = no authentication. Ownership rules apply on top (e.g. a doctor cancels only own prescriptions, BR-P-05).

Endpoint (under /api/v1 unless /actuator)	operationId	ADMIN	DOCTOR	PHARMACIST
POST `/auth/login`	`login`	public	public	public
POST `/auth/refresh`	`refreshToken`	public	public	public
POST `/auth/logout`	`logout`	public	public	public
GET `/auth/me`	`getCurrentUser`	yes	yes	yes
POST `/auth/change-password`	`changePassword`	yes	yes	yes
GET `/users`	`listUsers`	yes	-	-
POST `/users`	`createUser`	yes	-	-
GET `/users/{userId}`	`getUser`	yes	-	-
PUT `/users/{userId}`	`updateUser`	yes	-	-
POST `/users/{userId}/reset-password`	`resetUserPassword`	yes	-	-
GET `/staff`	`listStaff`	yes	yes	yes
GET `/patients`	`searchPatients`	-	yes	-
POST `/patients`	`createPatient`	-	yes	-
GET `/patients/by-number/{patientNumber}`	`getPatientByNumber`	-	yes	-
GET `/patients/{patientId}`	`getPatient`	-	yes	-
PUT `/patients/{patientId}`	`updatePatient`	-	yes	-
GET `/patients/{patientId}/medical-history`	`getMedicalHistory`	-	yes	-
POST `/consultations`	`createConsultation`	-	yes	-
GET `/consultations/{consultationId}`	`getConsultation`	-	yes	-
POST `/consultations/{consultationId}/prescriptions`	`addPrescriptionToConsultation`	-	yes	-

Endpoint (under /api/v1 unless /actuator)	operationId	ADMIN	DOCTOR	PHARMACIST
GET `/prescriptions`	`listPrescriptions`	-	yes	yes
GET `/prescriptions/{prescriptionId}`	`getPrescription`	-	yes	yes
POST `/prescriptions/{prescriptionId}/start`	`startPrescription`	-	-	yes
POST `/prescriptions/{prescriptionId}/dispense`	`dispensePrescription`	-	-	yes
POST `/prescriptions/{prescriptionId}/cancel`	`cancelPrescription`	-	yes	yes
GET `/medicines`	`listMedicines`	yes	yes	yes
POST `/medicines`	`createMedicine`	yes	-	yes
GET `/medicines/{medicineId}`	`getMedicine`	yes	yes	yes
PUT `/medicines/{medicineId}`	`updateMedicine`	yes	-	yes
GET `/queue-sessions`	`listQueueSessions`	-	yes	yes
POST `/queue-sessions`	`createQueueSession`	-	yes	yes
GET `/queue-sessions/{queueSessionId}`	`getQueueSession`	-	yes	yes
POST `/queue-sessions/{queueSessionId}/open`	`openQueueSession`	-	yes	yes
POST `/queue-sessions/{queueSessionId}/close`	`closeQueueSession`	-	yes	yes
GET `/queue-sessions/{queueSessionId}/entries`	`listQueueEntries`	-	yes	yes
POST `/queue-sessions/{queueSessionId}/entries`	`createBooking`	-	yes	yes
POST `/queue-sessions/{queueSessionId}/call-next`	`callNextPatient`	-	yes	-
GET `/queue-entries/{queueEntryId}`	`getQueueEntry`	-	yes	yes
POST `/queue-entries/{queueEntryId}/call`	`callQueueEntry`	-	yes	-
POST `/queue-entries/{queueEntryId}/complete`	`completeQueueEntry`	-	yes	-
POST `/queue-entries/{queueEntryId}/skip`	`skipQueueEntry`	-	yes	-
POST `/queue-entries/{queueEntryId}/requeue`	`requeueQueueEntry`	-	yes	yes
POST `/queue-entries/{queueEntryId}/cancel`	`cancelBooking`	-	yes	yes
GET `/public/queue-sessions`	`listPublicQueueSessions`	public	public	public
GET `/public/queue-sessions/{queueSessionId}`	`getPublicQueueStatus`	public	public	public
GET `/audit-events`	`listAuditEvents`	yes	-	-
GET `/actuator/health`	`health`	public	public	public
GET `/actuator/health/liveness`	`healthLiveness`	public	public	public

Endpoint (under /api/v1 unless /actuator)	operationId	ADMIN	DOCTOR	PHARMACIST
GET `/actuator/health/readiness`	`healthReadiness`	public	public	public

5. Functional requirements

5.1 Summary

ID	Area	Title	Priority
FR-001	Authentication	Staff login	MUST
FR-002	Authentication	Session refresh with rotation	MUST
FR-003	Authentication	Logout	MUST
FR-004	Authentication	Password change and forced change	MUST
FR-005	Authentication	Account lockout	MUST
FR-006	Authorization	Role-based access control	MUST
FR-007	Users	Staff account administration	MUST
FR-008	Users	Staff directory	SHOULD
FR-009	Users	Bootstrap administrator	MUST
FR-010	Authentication	Idle logout on shared workstations	MUST
FR-011	Patients	Look up patient by patient number	MUST
FR-012	Patients	Search patients	SHOULD
FR-013	Patients	Register new patient	MUST
FR-014	Patients	Unknown date of birth	MUST
FR-015	Patients	Update patient demographics	MUST
FR-016	Medical history	View medical history	MUST
FR-017	Consultations	Record a consultation	MUST
FR-018	Consultations	Multi-medicine prescription	MUST
FR-019	Consultations	Catalogue and free-text medicines	MUST
FR-020	Consultations	Atomic consultation submission	MUST
FR-021	Prescriptions	Automatic routing to the pharmacist queue	MUST
FR-022	Consultations	Submission confirmation and consultation detail	MUST
FR-023	Consultations	Immutable clinical history	MUST
FR-024	Consultations	Link consultation to queue entry	SHOULD
FR-025	Pharmacy	Pharmacist prescription queue	MUST
FR-026	Pharmacy	Prescription detail for dispensing	MUST
FR-027	Pharmacy	Mark in progress	SHOULD
FR-028	Pharmacy	Dispense with per-item record	MUST
FR-029	Pharmacy	Exactly-once dispensing	MUST
FR-030	Prescriptions	Cancel prescription	MUST
FR-031	Prescriptions	Add or re-issue a prescription	SHOULD
FR-032	Prescriptions	Doctor sees dispensing status	MUST
FR-033	Medicines	Medicine catalogue management	SHOULD

ID	Area	Title	Priority
FR-034	Queue	Create queue session	MUST
FR-035	Queue	Open and close session	MUST
FR-036	Bookings	Telephone and walk-in booking	MUST
FR-037	Bookings	Duplicate booking warning	SHOULD
FR-038	Bookings	Cancel booking	MUST
FR-039	Queue	Call next patient (doctor's queue button)	MUST
FR-040	Queue	Call a specific patient	SHOULD
FR-041	Queue	Complete, skip and requeue	MUST
FR-042	Queue	Staff queue status	MUST
FR-043	Public queue	Public list of today's queues	MUST
FR-044	Public queue	Public queue display	MUST
FR-045	Public queue	Position helper for patients	SHOULD
FR-046	Audit	Audit trail	MUST
FR-047	Audit	Audit log viewer	SHOULD
FR-048	Platform	Idempotent create operations	MUST
FR-049	Platform	Health checks	MUST
FR-050	Platform	Retention and housekeeping jobs	MUST

5.2 Detailed requirements and acceptance criteria

Each requirement lists its acceptance criteria and its trace to API operations, screens and tests. The full trace including database entities is the traceability matrix in section 17.

Authentication

> **FR-001 - Staff login** MUST
>
> Staff authenticate with username and password and receive a 15-minute access token and a 12-hour refresh session.
>
> *Acceptance criteria:*
>
> 1. Given valid credentials, POST /auth/login returns 200 with accessToken, expiresIn = 900 and user.roles, and sets the HttpOnly dms_refresh cookie.
> 2. Given an unknown username or wrong password, the response is 401 INVALID_CREDENTIALS with identical body and comparable latency (within 100 ms).
> 3. After login the frontend routes DOCTOR to /doctor/dashboard, PHARMACIST to /pharmacist/dashboard, ADMIN to /admin/users (first role in that order).
>
> API: `POST /api/v1/auth/login` · Screens: `/login` · Tests: BT-001, FT-001

FR-002 - Session refresh with rotation MUST

The frontend keeps the access token in memory only and obtains new ones from the refresh cookie; refresh tokens rotate on every use and reuse is detected.

Acceptance criteria:

1. Reloading a staff page silently restores the session via POST /auth/refresh while the family is younger than APP_JWT_REFRESH_TOKEN_TTL.
2. Two tabs refreshing at the same moment both end up authenticated (REFRESH_TOKEN_ROTATED is retried once).
3. Reusing a rotated token more than 30 s later returns 401 SESSION_REVOKED and revokes every token of that family.

API: `POST /api/v1/auth/refresh` · Screens: `/login` · Tests: BT-002, FT-002

FR-003 - Logout MUST

Logout revokes the refresh-token family, clears the cookie and clears all cached data in the browser.

Acceptance criteria:

1. After logout, POST /auth/refresh with the old cookie returns 401 REFRESH_TOKEN_INVALID.
2. After logout, the TanStack Query cache is empty and navigating Back does not show patient data.

API: `POST /api/v1/auth/logout` · Screens: `/change-password` · Tests: BT-002, FT-015

FR-004 - Password change and forced change MUST

New and reset accounts must change their password before using the system; users can change their password at any time.

Acceptance criteria:

1. A user with mustChangePassword = true receives 403 PASSWORD_CHANGE_REQUIRED on every endpoint except auth/me, auth/change-password and auth/logout, and the UI shows /change-password.
2. New passwords shorter than 12 characters, equal to the current password or containing the username are rejected.
3. A successful change revokes all other refresh-token families of the user.

API: `POST /api/v1/auth/change-password`, `GET /api/v1/auth/me` · Screens: `/change-password` · Tests: BT-004

FR-005 - Account lockout MUST

Five consecutive failed logins lock the account for 15 minutes.

Acceptance criteria:

1. The 5th consecutive failure sets locked_until = now + 15 min and writes AUTH_ACCOUNT_LOCKED.
2. During the lock a correct password returns 401 ACCOUNT_LOCKED with lockedUntil; a wrong password returns INVALID_CREDENTIALS.
3. A successful login resets the failure counter; an admin enabling the account clears the lock.

API: `POST /api/v1/auth/login` · Screens: `/login` · Tests: BT-001

Authorization

FR-006 - Role-based access control MUST

Every endpoint enforces the roles listed in openapi.yaml x-roles on the server; a user may hold several roles.

Acceptance criteria:

1. The authorization-matrix test derived from openapi.yaml passes for anonymous, ADMIN, DOCTOR and PHARMACIST callers.
2. A PHARMACIST calling a patient, history or consultation endpoint receives 403 FORBIDDEN and an ACCESS_DENIED audit event is written.
3. Hiding a button is never the only control - direct API calls are rejected identically.

API: `GET /api/v1/auth/me` · Screens: `/login` · Tests: BT-003, BT-035, FT-003

Users

FR-007 - Staff account administration MUST

ADMIN users create staff accounts, assign roles, enable/disable accounts and reset passwords.

Acceptance criteria:

1. Creating a user with an existing username returns 409 DUPLICATE_USERNAME.
2. Disabling a user revokes their refresh tokens; their next refresh returns 401 ACCOUNT_DISABLED.
3. An admin cannot disable themselves or remove their own ADMIN role, and the last enabled admin cannot be removed.

API: `GET /api/v1/users`, `POST /api/v1/users`, `GET /api/v1/users/{userId}`, `PUT /api/v1/users/{userId}`, `POST /api/v1/users/{userId}/reset-password` · Screens: `/admin/users`, `/admin/users/[userId]` · Tests: BT-032

FR-008 - Staff directory SHOULD

Staff can list enabled colleagues (id, name, roles only), e.g. to assign a doctor to a queue session.

Acceptance criteria:

1. GET /staff?role=DOCTOR returns only enabled users holding DOCTOR, without usernames or registration numbers.

API: `GET /api/v1/staff` · Screens: `/queue`, `/admin/audit-events` · Tests: BT-003

FR-009 - Bootstrap administrator MUST

On first start with an empty users table, an ADMIN is created from APP_BOOTSTRAP_ADMIN_USERNAME / APP_BOOTSTRAP_ADMIN_PASSWORD with mustChangePassword = true.

Acceptance criteria:

1. Starting with users present never creates or modifies an account, even if the variables are set.
2. Starting with an empty table and no variables logs a warning and creates nothing.

API: (no endpoint) · Screens: - · Tests: BT-032

Authentication

FR-010 - Idle logout on shared workstations MUST

The staff UI logs out after NEXT_PUBLIC_IDLE_TIMEOUT_MINUTES (default 30) without keyboard, pointer or touch activity.

Acceptance criteria:

1. After the timeout the UI calls POST /auth/logout, clears the cache and shows /login with a "signed out due to inactivity" message.
2. Background polling does not count as activity.

API: `POST /api/v1/auth/logout` · Screens: `/doctor/dashboard`, `/pharmacist/prescriptions` · Tests: FT-015

Patients

FR-011 - Look up patient by patient number MUST

A doctor enters a patient number; the system returns the patient or a clear "not found" that offers registration.

Acceptance criteria:

1. Lookup is case-insensitive and ignores surrounding spaces (" p-0001 " finds P-0001).
2. An unknown number returns 404 PATIENT_NOT_FOUND and the UI offers "Register new patient" with the number pre-filled.
3. Every successful lookup writes PATIENT_VIEWED with the patient id.

API: `GET /api/v1/patients/by-number/{patientNumber}` · Screens: `/doctor/patients/search`, `/doctor/dashboard` · Tests: BT-005, FT-004

FR-012 - Search patients SHOULD

A doctor can search by name fragment, phone number or patient-number prefix to find patients without a card and to avoid duplicate registrations.

Acceptance criteria:

1. Search requires 2-100 characters and returns at most 100 results per page sorted by name.
2. Results contain summary fields only (no allergies or address).
3. Each search writes PATIENT_SEARCHED with the term and result count.

API: `GET /api/v1/patients` · Screens: `/doctor/patients/search` · Tests: FT-004, BT-026

FR-013 - Register new patient MUST

A doctor registers a patient with a unique patient number, name, sex, date of birth (or estimated age) and optional phone, address and allergies.

Acceptance criteria:

1. A duplicate patient number returns 409 DUPLICATE_PATIENT_NUMBER with existingPatientId, including when two registrations race.
2. The stored patient number is upper-case and immutable.
3. A retried submission with the same Idempotency-Key returns the same patient (201) and creates no second row.

API: `POST /api/v1/patients` · Screens: `/doctor/patients/new` · Tests: BT-005, BT-006, BT-010, FT-005

FR-014 - Unknown date of birth MUST

When the date of birth is unknown, staff enter an estimated age; the system stores a derived date of birth flagged as estimated and always computes age.

Acceptance criteria:

1. Supplying both dateOfBirth and estimatedAgeYears, or neither, returns 400 VALIDATION_FAILED.
2. estimatedAgeYears = 70 entered on 2026-10-12 stores dateOfBirth 1956-10-12 with dateOfBirthEstimated = true; ageYears is 70 that day and 71 on 2027-10-12.
3. The UI displays estimated ages as "~70 y (estimated)".

API: `POST /api/v1/patients`, `PUT /api/v1/patients/{patientId}` · Screens: `/doctor/patients/new`, `/doctor/patients/[patientId]` · Tests: BT-005, BT-033, FT-005

FR-015 - Update patient demographics MUST

A doctor can correct name, sex, date of birth, phone, address and known allergies; concurrent edits are detected.

Acceptance criteria:

1. A PUT with a stale version returns 409 CONCURRENT_MODIFICATION and changes nothing.
2. The patient number cannot be changed through the API.
3. PATIENT_UPDATED lists changed field names but not values.

API: `PUT /api/v1/patients/{patientId}`, `GET /api/v1/patients/{patientId}` · Screens: `/doctor/patients/[patientId]` · Tests: BT-007

Medical history

FR-016 - View medical history MUST

A doctor sees the patient's demographics, previous conditions, all previous consultations (newest first) with diagnoses, notes, prescriptions, prescribed medicines and dispensing status.

Acceptance criteria:

1. Consultations are paginated (default 10 per page) and ordered by consultationAt descending.
2. Cancelled and superseded prescriptions are shown with their status; dispensed prescriptions show per-item dispensed quantities and the pharmacist.
3. Loading one page executes a bounded number of SQL statements independent of the number of consultations (no N+1).

API: `GET /api/v1/patients/{patientId}/medical-history` · Screens: `/doctor/patients/[patientId]` · Tests: BT-008, FT-008

Consultations

FR-017 - Record a consultation MUST

A doctor records a consultation for an existing or newly registered patient with presenting complaint, clinical notes and 1-10 diagnoses/conditions.

Acceptance criteria:

1. The consultation's doctor and timestamp are taken from the authenticated user and server clock.
2. Creating a consultation never modifies earlier consultations.

API: `POST /api/v1/consultations` · Screens: `/doctor/consultations/new` · Tests: BT-009, FT-006

FR-018 - Multi-medicine prescription MUST

A consultation can include one prescription with 1-30 items, each with medicine, strength, form, dose, frequency, duration, quantity, unit and instructions.

Acceptance criteria:

1. Items keep the order entered (lineNumber 1..n).
2. Quantity accepts up to 2 decimals (e.g. 7.5) and is stored as NUMERIC(7,2) / BigDecimal.
3. An advice-only consultation without prescription is accepted.

API: `POST /api/v1/consultations` · Screens: `/doctor/consultations/new` · Tests: BT-011, FT-006

FR-019 - Catalogue and free-text medicines MUST

Items reference a catalogue medicine or a free-text name; inactive catalogue medicines cannot be prescribed.

Acceptance criteria:

1. The prescription editor's autocomplete only offers active medicines.
2. Submitting an inactive medicineId returns 422 MEDICINE_INACTIVE pointing at the item.
3. Catalogue name, strength and form are snapshotted into the item; later catalogue edits do not change history.

API: `POST /api/v1/consultations`, `GET /api/v1/medicines` · Screens: `/doctor/consultations/new` · Tests: BT-011, FT-006

FR-020 - Atomic consultation submission MUST

Consultation, diagnoses, prescription and items are saved in one transaction; any failure saves nothing.

Acceptance criteria:

1. Injecting a failure on the last item leaves zero new rows in consultations, consultation_diagnoses, prescriptions and prescription_items.
2. On a network error the form keeps all entered data and a retry uses the same Idempotency-Key.

API: `POST /api/v1/consultations` · Screens: `/doctor/consultations/new` · Tests: BT-009, BT-010, BT-027, FT-007, FT-016

Prescriptions

FR-021 - Automatic routing to the pharmacist queue MUST

A submitted prescription is immediately in status PENDING and visible in the pharmacist queue without any extra action.

Acceptance criteria:

1. Within one polling interval (default 10 s) after submission the prescription appears at /pharmacist/prescriptions.

API: `POST /api/v1/consultations`, `GET /api/v1/prescriptions` · Screens: `/doctor/consultations/new`, `/pharmacist/prescriptions` · Tests: BT-009, FT-009, FT-017

Consultations

FR-022 - Submission confirmation and consultation detail MUST

After submission the doctor sees a confirmation with the prescription number and live dispensing status.

Acceptance criteria:

1. The confirmation shows RX number, items and status, and refreshes until the status is DISPENSED or CANCELLED.

API: `GET /api/v1/consultations/{consultationId}` · Screens: `/doctor/consultations/[consultationId]` · Tests: FT-008

FR-023 - Immutable clinical history MUST

Consultations, diagnoses and prescription items cannot be edited or deleted; corrections are made by cancelling and re-issuing prescriptions or by a new consultation.

Acceptance criteria:

1. openapi.yaml exposes no PUT/PATCH/DELETE for consultations, diagnoses or prescription items, and the contract test confirms the implementation exposes none.

API: (no endpoint) · Screens: - · Tests: BT-029

FR-024 - Link consultation to queue entry SHOULD

When a consultation is started from the queue console, it records the queue entry being served.

Acceptance criteria:

1. A queue entry can be linked to at most one consultation (409 QUEUE_ENTRY_ALREADY_LINKED).
2. Linking does not change the queue entry status.

API: `POST /api/v1/consultations`, `GET /api/v1/queue-entries/{queueEntryId}` · Screens: `/doctor/consultations/new` · Tests: BT-009

Pharmacy

FR-025 - Pharmacist prescription queue MUST

Pharmacists see prescriptions filtered by status (default PENDING and IN_PROGRESS), oldest first, refreshed automatically.

Acceptance criteria:

1. Tabs Pending, In progress, Dispensed and Cancelled (the last two limited to prescriptions submitted today) show totals and lists.
2. The list refreshes every NEXT_PUBLIC_PRESCRIPTION_POLL_INTERVAL_MS while the tab is visible.

API: `GET /api/v1/prescriptions` · Screens: `/pharmacist/prescriptions`, `/pharmacist/dashboard` · Tests: FT-009, BT-026

FR-026 - Prescription detail for dispensing MUST

The pharmacist sees patient name, number, sex, age, known allergies, prescribing doctor, consultation date, submission time and all items - but no diagnoses or clinical notes.

Acceptance criteria:

1. GET /prescriptions/{id} contains no diagnosis or clinical-note fields (schema-enforced).
2. Opening a prescription writes PRESCRIPTION_VIEWED.

API: `GET /api/v1/prescriptions/{prescriptionId}` · Screens: `/pharmacist/prescriptions/[prescriptionId]` · Tests: BT-035

FR-027 - Mark in progress SHOULD

A pharmacist marks a pending prescription IN_PROGRESS so colleagues see it is being prepared.

Acceptance criteria:

1. Repeating the action is a no-op (200); starting a DISPENSED or CANCELLED prescription returns 409 INVALID_STATUS_TRANSITION.

API: `POST /api/v1/prescriptions/{prescriptionId}/start` · Screens: `/pharmacist/prescriptions/[prescriptionId]` · Tests: BT-012

FR-028 - Dispense with per-item record MUST

The pharmacist confirms what was handed over for each item; the system records quantities, item status, pharmacist and timestamp and sets the prescription to DISPENSED.

Acceptance criteria:

1. The request must contain every item exactly once with 0 <= quantity <= prescribed.
2. Items below the prescribed quantity require a note; all-zero requests are rejected with NOTHING_DISPENSED.
3. The outcome is FULL only when every item is fully dispensed, otherwise PARTIAL.

API: `POST /api/v1/prescriptions/{prescriptionId}/dispense` · Screens: `/pharmacist/prescriptions/[prescriptionId]` · Tests: BT-014, FT-010

FR-029 - Exactly-once dispensing MUST

Repeated clicks, retries or concurrent pharmacists can never create more than one dispensing record, and a dispensed prescription never returns to PENDING.

Acceptance criteria:

1. 10 concurrent dispense requests produce exactly one dispensings row.
2. A retry with the same Idempotency-Key returns the original 200 response.

API: `POST /api/v1/prescriptions/{prescriptionId}/dispense` · Screens: `/pharmacist/prescriptions/[prescriptionId]` · Tests: BT-013, BT-010

Prescriptions

FR-030 - Cancel prescription MUST

The prescribing doctor or a pharmacist can cancel a PENDING or IN_PROGRESS prescription with a mandatory reason.

Acceptance criteria:

1. Another doctor receives 403 NOT_RESOURCE_OWNER.
2. DISPENSED prescriptions cannot be cancelled (409).

API: `POST /api/v1/prescriptions/{prescriptionId}/cancel` · Screens: `/doctor/consultations/[consultationId]`, `/pharmacist/prescriptions/[prescriptionId]` · Tests: BT-015, BT-012

FR-031 - Add or re-issue a prescription SHOULD

Within 24 hours of a consultation its doctor can add a prescription (if none is active), optionally superseding a cancelled one.

Acceptance criteria:

1. A consultation never has more than one non-cancelled prescription (database-enforced).
2. The superseded prescription shows supersededByPrescriptionId.

API: `POST /api/v1/consultations/{consultationId}/prescriptions` · Screens: `/doctor/consultations/[consultationId]` · Tests: BT-016

FR-032 - Doctor sees dispensing status MUST

Doctors see the dispensing status of their prescriptions in the consultation view, the patient history and a "My prescriptions" list.

Acceptance criteria:

1. A dispensing confirmed by the pharmacist is visible to the doctor within one polling interval.
2. DOCTOR callers of GET /prescriptions only ever receive their own prescriptions.

API: `GET /api/v1/prescriptions`, `GET /api/v1/consultations/{consultationId}`, `GET /api/v1/patients/{patientId}/medical-history` · Screens: `/doctor/prescriptions`, `/doctor/consultations/[consultationId]`, `/doctor/patients/[patientId]` · Tests: FT-008, BT-003

Medicines

FR-033 - Medicine catalogue management SHOULD

Pharmacists and admins add, edit and (de)activate catalogue medicines. There is no stock or inventory in release 1.0.

Acceptance criteria:

1. Name + strength + form are unique case-insensitively (409 DUPLICATE_MEDICINE).
2. Deactivated medicines disappear from the prescription autocomplete but existing prescriptions are unaffected.

API: `GET /api/v1/medicines`, `POST /api/v1/medicines`, `GET /api/v1/medicines/{medicineId}`, `PUT /api/v1/medicines/{medicineId}` · Screens: `/medicines` · Tests: BT-034

Queue

FR-034 - Create queue session MUST

Staff create a named queue session for a date (today .. +30 days), optionally assigned to a doctor; each session has its own numbering starting at 1.

Acceptance criteria:

1. Session names are unique per date case-insensitively (409 DUPLICATE_QUEUE_SESSION).
2. A past date or a date more than 30 days ahead returns 422 INVALID_SESSION_DATE.

API: `POST /api/v1/queue-sessions`, `GET /api/v1/queue-sessions`, `GET /api/v1/staff` · Screens: `/queue` · Tests: BT-021

FR-035 - Open and close session MUST

A session is opened on its date to allow calling and closed at the end of the clinic; force-closing cancels waiting entries and completes served ones; past sessions are auto-closed nightly.

Acceptance criteria:

1. Closing with waiting patients and force = false returns 409 QUEUE_SESSION_HAS_ACTIVE_ENTRIES with counts.
2. A closed session accepts no bookings or calls (409 QUEUE_SESSION_CLOSED).

API: `POST /api/v1/queue-sessions/{queueSessionId}/open`, `POST /api/v1/queue-sessions/{queueSessionId}/close` · Screens: `/queue`, `/queue/[queueSessionId]` · Tests: BT-021

Bookings

FR-036 - Telephone and walk-in booking MUST

Staff register a booking for a caller or walk-in and receive the next queue number to tell the patient.

Acceptance criteria:

1. Numbers are 1..999 per session, unique, gap-free and never reused - verified with 100 concurrent requests.
2. A retried request with the same Idempotency-Key returns the same number.
3. The new number is visible on the public display within one polling interval.

API: `POST /api/v1/queue-sessions/{queueSessionId}/entries` · Screens: `/bookings/new`, `/queue/[queueSessionId]` · Tests: BT-017, BT-010, FT-011

FR-037 - Duplicate booking warning SHOULD

The system warns when the same phone number already holds an active booking in the session; staff can confirm a deliberate duplicate.

Acceptance criteria:

1. The 409 response names the existing queue number; confirming re-submits with allowDuplicate = true.

API: `POST /api/v1/queue-sessions/{queueSessionId}/entries` · Screens: `/bookings/new` · Tests: BT-018, FT-011

FR-038 - Cancel booking MUST

Staff cancel a waiting booking with a reason; the number is not reused.

Acceptance criteria:

1. Only WAITING entries can be cancelled; others return 409 INVALID_STATUS_TRANSITION.

API: `POST /api/v1/queue-entries/{queueEntryId}/cancel` · Screens: `/queue/[queueSessionId]` · Tests: BT-020

Queue

FR-039 - Call next patient (doctor's queue button) MUST

The doctor's "Next patient" button completes the current patient (optional) and calls the lowest waiting number atomically; it never creates bookings and can never skip a patient through double clicks or concurrent presses.

Acceptance criteria:

1. Two concurrent presses based on the same displayed number result in exactly one advance; the other receives 409 QUEUE_STATE_CHANGED and the UI refreshes.
2. With nobody waiting the button shows "No patients waiting" and nothing is advanced.

API: `POST /api/v1/queue-sessions/{queueSessionId}/call-next` · Screens: `/doctor/dashboard`, `/queue/[queueSessionId]` · Tests: BT-019, FT-012

FR-040 - Call a specific patient SHOULD

A doctor can call a specific waiting entry out of order (emergency, priority).

Acceptance criteria:

1. The called number becomes the public "now serving" number.

API: `POST /api/v1/queue-entries/{queueEntryId}/call` · Screens: `/queue/[queueSessionId]` · Tests: BT-020

FR-041 - Complete, skip and requeue MUST

The doctor marks a served patient completed or absent (skipped); staff return a late patient to the waiting list keeping the original number.

Acceptance criteria:

1. Transitions follow the queue-entry state machine; invalid transitions return 409 INVALID_STATUS_TRANSITION with currentStatus.
2. A requeued entry is the next one called by call-next.

API: `POST /api/v1/queue-entries/{queueEntryId}/complete`, `POST /api/v1/queue-entries/{queueEntryId}/skip`, `POST /api/v1/queue-entries/{queueEntryId}/requeue` · Screens: `/queue/[queueSessionId]`, `/doctor/dashboard` · Tests: BT-020

FR-042 - Staff queue status MUST

Doctors and pharmacists see current serving number, next number, counts by status and the entry list, refreshed automatically.

Acceptance criteria:

1. The staff view refreshes every NEXT_PUBLIC_QUEUE_POLL_INTERVAL_MS using ETag (304 when unchanged).

API: `GET /api/v1/queue-sessions/{queueSessionId}`, `GET /api/v1/queue-sessions/{queueSessionId}/entries` · Screens: `/doctor/dashboard`, `/queue/[queueSessionId]` · Tests: FT-012, BT-023

Public queue

FR-043 - Public list of today's queues MUST

Anyone can open /public/queue to choose one of today's queues without logging in.

Acceptance criteria:

1. The list contains only label, status, now-serving number and waiting count.

API: `GET /api/v1/public/queue-sessions` · Screens: `/public/queue` · Tests: BT-022

FR-044 - Public queue display MUST

A public page shows the number being served and waiting numbers, updating automatically, with no personal or clinical data.

Acceptance criteria:

1. The page updates within NFR-004 latency without manual refresh.
2. The JSON response contains only the PublicQueueStatus fields; names, phones, patient numbers and notes never appear (key-whitelist test).

API: `GET /api/v1/public/queue-sessions/{queueSessionId}` · Screens: `/public/queue/[queueSessionId]` · Tests: BT-022, BT-023, FT-013

FR-045 - Position helper for patients SHOULD

A patient can type their own number on the public page to see how many numbers are ahead; the number is never sent to the server.

Acceptance criteria:

1. Entering 7 while waitingNumbers = [5, 6, 7, 9] shows "2 ahead of you".

API: `GET /api/v1/public/queue-sessions/{queueSessionId}` · Screens: `/public/queue/[queueSessionId]` · Tests: FT-013

Audit

FR-046 - Audit trail MUST

Security events, all clinical and queue writes, and views of patient records, histories, consultations and prescriptions are written to an append-only audit trail.

Acceptance criteria:

1. Every action listed in the AuditAction catalogue is produced by its trigger with actor, entity, patient id, outcome, request id and IP.
2. Writes are audited in the same transaction as the change; a rolled-back change leaves no audit row.
3. audit_events rows cannot be updated or deleted (trigger).

API: (no endpoint) · Screens: - · Tests: BT-024

FR-047 - Audit log viewer SHOULD

Admins filter the audit trail by patient, user, action, entity and time range.

Acceptance criteria:

1. Filtering by patientId lists everyone who viewed or changed that patient's records, newest first.

API: `GET /api/v1/audit-events` · Screens: `/admin/audit-events` · Tests: BT-024

Platform

FR-048 - Idempotent create operations MUST

createPatient, createConsultation, addPrescriptionToConsultation, createBooking and dispensePrescription require an Idempotency-Key and replay the original result for retries.

Acceptance criteria:

1. Missing key → 400 IDEMPOTENCY_KEY_REQUIRED; reused key with a different body → 422 IDEMPOTENCY_KEY_REUSED.

API: `POST /api/v1/patients`, `POST /api/v1/consultations`, `POST /api/v1/consultations/{consultationId}/prescriptions`, `POST /api/v1/queue-sessions/{queueSessionId}/entries`, `POST /api/v1/prescriptions/{prescriptionId}/dispense` · Screens: `/doctor/patients/new`, `/doctor/consultations/new`, `/doctor/consultations/[consultationId]`, `/bookings/new`, `/pharmacist/prescriptions/[prescriptionId]` · Tests: BT-010, FT-007

FR-049 - Health checks MUST

The backend exposes liveness, readiness (including database) and aggregate health without details.

Acceptance criteria:

1. With the database stopped, /actuator/health/readiness returns 503 and /actuator/health/liveness returns 200.

API: `GET /actuator/health`, `GET /actuator/health/liveness`, `GET /actuator/health/readiness` · Screens: - · Tests: BT-029, FT-017

FR-050 - Retention and housekeeping jobs MUST

A nightly job anonymises queue PII older than APP_RETENTION_QUEUE_PII_DAYS, auto-closes past sessions, and deletes expired refresh tokens (> 7 days) and idempotency keys.

Acceptance criteria:

1. Running the job twice in a row changes nothing the second time (idempotent) and writes RETENTION_JOB_EXECUTED each run.

API: (no endpoint) · Screens: - · Tests: BT-031, BT-021

6. Non-functional requirements

6.1 Non-functional requirements table

ID	Category	Requirement	Target	Verification	Tests
NFR-001	Performance	Staff read latency	Proposed target: p95 <= 500 ms, p99 <= 1 s for patient lookup, medical history (10 consultations), prescription list and queue session at 25 concurrent staff users on 2 vCPU / 4 GB backend + 2 vCPU / 4 GB PostgreSQL with 50 000 patients, 500 000 consultations.	k6 script backend/perf/staff-reads.js against the perf dataset (backend/perf/generate-perf-data.sql), 10-minute run, results archived in CI.	-
NFR-002	Performance	Staff write latency	Proposed target: p95 <= 800 ms for createConsultation (5 items), dispensePrescription, createBooking and callNextPatient under the NFR-001 load.	k6 script perf/staff-writes.js.	-
NFR-003	Performance	Public queue throughput	Proposed target: GET /public/queue-sessions/{id} sustains 50 req/s (= 250 displays/phones polling every 5 s) with p95 <= 150 ms; 304 responses p95 <= 50 ms.	k6 script perf/public-queue.js with If-None-Match.	BT-023
NFR-004	Performance	Queue update propagation	Proposed target: 95% of queue changes visible on the public display within 7 s (5 s poll + latency); 100% within 12 s.	Playwright test measures time from call-next response to display change (FT-017).	FT-013, FT-017
NFR-005	Availability	Service availability	Proposed target: 99.5% monthly availability during dispensary opening hours, excluding announced maintenance.	External uptime probe on /actuator/health/readiness every 60 s; monthly report.	-
NFR-006	Recoverability	Backup and restore	Proposed target: RPO <= 15 min (continuous WAL archiving + daily base backup), RTO <= 4 h; backups encrypted, retained 35 days, one copy off-site.	Quarterly restore drill into a clean server, documented in the ops runbook.	-
NFR-007	Security	Transport security	All production traffic over HTTPS (TLS 1.2+); HTTP redirects to HTTPS; HSTS max-age 31536000; database connections inside a private network or TLS.	testssl.sh / SSL Labs grade A against the production host.	-
NFR-008	Security	Credential storage	Passwords hashed with BCrypt cost 12 via DelegatingPasswordEncoder; refresh tokens stored only as SHA-256; no secrets in source, images or logs.	Unit test asserts {bcrypt} prefix and cost; secret scanning (gitleaks) in CI.	BT-001, BT-028
NFR-009	Security	Session security	Access token 15 min, refresh family 12 h absolute, rotation on every refresh, reuse detection; cookie HttpOnly, SameSite=Strict, Secure, Path=/api/v1/auth; client idle logout 30 min.	BT-002, FT-015 and manual cookie inspection.	BT-002, FT-015
NFR-010	Privacy	Log privacy	Logs never contain passwords, tokens, Authorization/Cookie headers, clinical notes, diagnoses, medicine lists, patient names or phone numbers; identifiers (UUIDs) and request ids only.	Log-capture test over representative flows (BT-028); code review checklist.	BT-028
NFR-011	Privacy	Public data minimisation	Unauthenticated endpoints return only PublicQueueStatus / PublicQueueSessionSummary fields.	JSON key-whitelist test with seeded PII (BT-022).	BT-022
NFR-012	Privacy	Least privilege for clinical data	Pharmacists cannot read diagnoses, clinical notes or full histories; admins cannot read any clinical data; every denied attempt is audited.	BT-003 matrix + BT-035.	BT-003, BT-035

ID	Category	Requirement	Target	Verification	Tests
NFR-013	Security	Injection and input handling	All SQL via JPA/parameterised queries (no string concatenation); all input validated server-side; React escaping only (no dangerouslySetInnerHTML).	Static analysis (SpotBugs + find-sec-bugs, eslint-plugin-security) with zero high findings; OWASP ZAP baseline scan with no high alerts.	BT-005, BT-011
NFR-014	Security	Security headers	Frontend sends Content-Security-Policy (default-src 'self'; frame-ancestors 'none'), X-Content-Type-Options nosniff, Referrer-Policy no-referrer, Permissions-Policy minimal; API sends Cache-Control no-store on authenticated responses.	Automated header check in FT-017 and ZAP baseline.	FT-017
NFR-015	Accessibility	Accessibility	WCAG 2.2 level AA for all screens; full keyboard operation; visible focus; colour is never the only status indicator (badges have text).	axe-core via Playwright with zero serious/ critical violations (FT-014) + manual screen-reader smoke test (NVDA/ VoiceOver) per release.	FT-014
NFR-016	Usability	Responsive layout and browsers	Usable from 360 px to 1920 px width; latest two versions of Chrome, Edge, Firefox and Safari; public display legible at 5 m on a 1080p screen (now-serving number >= 160 px high).	FT-018 viewport tests; manual check on a waiting-room TV.	FT-018
NFR-017	Integrity	Transactional integrity	Each API write is one database transaction; invariants (unique numbers, one dispensing per prescription, one active prescription per consultation, item ownership) are enforced by database constraints, not only code.	BT-009, BT-013, BT-017 plus schema constraint tests.	BT-009, BT-013, BT-017
NFR-018	Concurrency	Concurrency correctness	No duplicate or skipped queue numbers and no duplicate dispensing under 100 concurrent requests; lock waits bounded by 5 s lock_timeout (then 503 SERVICE_UNAVAILABLE, retryable).	Concurrency integration tests on PostgreSQL Testcontainers.	BT-017, BT-019, BT-013
NFR-019	Operability	Observability	Structured JSON logs (ECS) with requestId and userId; X-Request-Id propagated; Actuator health and Prometheus metrics (/actuator/prometheus restricted to private-network callers, not routed publicly).	Log format test; metrics endpoint reachable only internally.	BT-027
NFR-020	Operability	Configuration and secrets	All configuration via the documented environment variables; startup fails fast if a required variable is missing or APP_JWT_SECRET decodes to < 32 bytes.	Startup test with missing/short secret.	BT-025
NFR-021	Maintainability	Code quality and coverage	Modular monolith with package-per-feature; no business logic in controllers; JaCoCo line coverage >= 80% for service packages; frontend Vitest coverage >= 70% lines for features/ and lib/.	CI coverage gates; ArchUnit rules for layering.	-
NFR-022	Interoperability	API contract conformance	The running API conforms to openapi.yaml (paths, methods, status codes, schemas); frontend types are generated from openapi.yaml.	BT-029 contract test; frontend type generation check in CI (git diff --exit-code).	BT-029
NFR-023	Scalability	Horizontal scalability readiness	Backend is stateless (no HTTP session, no in-memory business state); two instances behind a load balancer pass the full test suite; only rate-limit counters are per-instance.	Compose profile with two backend replicas runs FT-017.	FT-017

ID	Category	Requirement	Target	Verification	Tests
NFR-024	Correctness	Time handling	Instants stored as TIMESTAMPTZ UTC and returned with Z; business dates use APP_TIMEZONE; all time-dependent logic uses an injected java.time.Clock.	BT-033 boundary tests.	BT-033
NFR-025	Security	Rate limiting	Login limited to APP_RATE_LIMIT_LOGIN_PER_MINUTE per IP, public endpoints to APP_RATE_LIMIT_PUBLIC_PER_MINUTE per IP; excess → 429 with Retry-After.	BT-030.	BT-030
NFR-026	Integrity	Audit integrity	audit_events is append-only (trigger); production DB role for the app has no TRUNCATE on audit_events; audit retention >= 7 years unless the owner sets otherwise.	BT-024; DB privilege review at deployment.	BT-024
NFR-027	Portability	Local start-up	docker compose up --build starts database, backend and frontend with seed data in <= 5 minutes on a developer laptop (8 GB RAM), with no manual steps beyond copying .env.example to .env.	CI job runs compose and FT-017 smoke test.	FT-017
NFR-028	Usability	Efficient key tasks	Doctor advances the queue with 1 click; pharmacist dispenses a fully available prescription with at most 3 interactions (open, confirm quantities, confirm); booking requires at most 3 fields.	Playwright step counts in FT-017.	FT-017
NFR-029	Security	Dependency hygiene	No known critical/high vulnerabilities in dependencies at release (OWASP dependency-check for Maven, npm audit --audit-level=high).	CI gate.	-
NFR-030	Usability	Localisation readiness	Release 1.0 UI is English; all user-visible strings live in one messages module so Sinhala/Tamil can be added without code changes to components.	Lint rule forbidding string literals in JSX outside messages (eslint i18n rule, warn level).	-

6.2 How performance targets are tested

The performance targets (NFR-001 to NFR-004) are **proposed targets** sized for a single dispensary (A-08) with headroom. They are tested with k6 scripts in `backend/perf/` against a dataset generated by `backend/perf/generate-perf-data.sql` (50 000 patients, 500 000 consultations, 1 000 000 prescription items, 60 days of queue data), on the reference hardware stated in NFR-001, with the Spring `prod` profile. A test passes when the p95/p99 thresholds hold for the full 10-minute steady-state phase and the error rate is below 0.1%. The scripts and dataset generator are part of the backend deliverable (backend-requirements.md section 13).

7. Business rules

7.1 General

- **BR-G-01** Instants are stored in UTC (`TIMESTAMPTZ`) and returned as ISO-8601 with `Z`. Business dates (session date, date of birth, "today") are evaluated in `APP_TIMEZONE` (default Asia/Colombo).
- **BR-G-02** Nothing clinical is hard-deleted in release 1.0: patients, consultations, prescriptions and dispensings have no delete operation; users are disabled, medicines deactivated.
- **BR-G-03** The system records and manages clinician-entered information only. It performs no automated diagnosis, dose calculation, interaction or allergy checking and makes no medicine recommendations. Known allergies are displayed prominently but not evaluated.
- **BR-G-04** Every write is attributed to the authenticated user and the server clock; clients cannot supply actor ids or timestamps.
- **BR-G-05** Repeating a state transition whose target state is already the current state is a no-op that returns 200 with the unchanged resource (no audit row, no version change). Any other transition not listed in 7.8 returns `409 INVALID_STATUS_TRANSITION`.
- **BR-G-06** Create operations that would be harmful if duplicated require an `Idempotency-Key` (FR-048).

7.2 Patients

- **BR-PAT-01** `patientNumber` is unique, trimmed, upper-cased, matches `^[A-Z0-9][A-Z0-9-]{2,19}$` and cannot be changed after registration.
- **BR-PAT-02** Exactly one of `dateOfBirth` or `estimatedAgeYears` is supplied. An estimated age N stores `dateOfBirth = today - N years` and `dateOfBirthEstimated = true`.
- **BR-PAT-03** Age is computed in completed years at today's date; it is never stored.
- **BR-PAT-04** Only doctors register and update patients in release 1.0.
- **BR-PAT-05** The registration screen runs a name search and shows possible matches before saving, to discourage duplicate patients (duplicates by name are allowed; duplicates by number are not).
- **BR-PAT-06** Changes to known allergies are audited (field name only).

7.3 Consultations

- **BR-C-01** A consultation is insert-only; the medical history is the ordered set of consultations. Creating one never modifies another.
- **BR-C-02** A consultation has 1-10 diagnoses (line order preserved) and at most one non-cancelled prescription.
- **BR-C-03** The consultation's doctor is the authenticated user; its time is the server time of submission.
- **BR-C-04** An optional queue entry link must point to an entry in status SERVING or COMPLETED; an entry can be linked to only one consultation; linking does not change the entry.
- **BR-C-05** Consultation, diagnoses, prescription and items are saved in one transaction; any failure saves nothing.

7.4 Prescriptions

- **BR-P-01** Lifecycle: created as `PENDING`; `PENDING -> IN_PROGRESS`, `PENDING|IN_PROGRESS -> DISPENSED`, `PENDING|IN_PROGRESS -> CANCELLED`. `DISPENSED` and `CANCELLED` are terminal; a prescription never returns to `PENDING`.
- **BR-P-02** A prescription is visible in the pharmacist queue as soon as it is created - there is no separate "send" step.

- **BR-P-03** 1-30 items. An item references a catalogue medicine (`medicineId`) **or** names a free-text medicine (`medicineName`), never both. Catalogue name, strength and form are copied (snapshot) into the item.
- **BR-P-04** An inactive catalogue medicine cannot be prescribed (`422 MEDICINE_INACTIVE`).
- **BR-P-05** Cancellation requires a reason (3-500 chars) and is allowed for the doctor who recorded the consultation and for any pharmacist.
- **BR-P-06** A doctor may add a prescription to their own consultation within 24 h of `consultationAt` when it has no active prescription, optionally superseding a cancelled one (re-issue). A prescription can be superseded at most once.
- **BR-P-07** Prescriptions and their items are immutable after submission; corrections are made by cancel + re-issue.
- **BR-P-08** In the prescription list doctors see only prescriptions of their own consultations; all doctors see all prescriptions of a patient through the medical history.
- **BR-P-09** The human-readable number is `RX-` + 6-digit zero-padded sequence (`RX-000123`). Gaps in the sequence are harmless.

7.5 Dispensing

- **BR-D-01** Only pharmacists dispense; allowed from `PENDING` or `IN_PROGRESS`.
- **BR-D-02** The dispense request lists every item exactly once with `0 <= quantityDispensed <= prescribed quantity` (max 2 decimals). Below the prescribed quantity a note is mandatory.
- **BR-D-03** At least one item must have a quantity above zero; if nothing can be supplied the prescription is cancelled instead.
- **BR-D-04** Item status (`FULL`, `PARTIAL`, `NOT_DISPENSED`) and outcome (`FULL`, `PARTIAL`) are derived by the server.
- **BR-D-05** A prescription has at most one dispensing record (database-enforced). Dispensing is irreversible in release 1.0.
- **BR-D-06** Any pharmacist may continue a prescription another pharmacist started.
- **BR-D-07** Remaining quantities after a partial dispensing are not tracked for later collection in release 1.0.
- **BR-D-08** A user holding both DOCTOR and PHARMACIST may dispense a prescription they wrote; the dispensing is audited as usual (confirm Q-04).
- **BR-D-09** Dispensing does not change any stock (no inventory).

7.6 Queue

- **BR-Q-01** A queue session is a named, numbered queue for one date; names are unique per date (case-insensitive). Sessions are created for today up to 30 days ahead.
- **BR-Q-02** Queue numbers are assigned per session starting at 1, unique, gap-free and never reused - including after cancellation. Maximum 999 per session.
- **BR-Q-03** Numbering restarts at 1 in each new session; because sessions belong to one date, it restarts at least daily.
- **BR-Q-04** Bookings are accepted while a session is SCHEDULED or OPEN. Calling, completing and skipping require OPEN. A session can be opened only on its date.
- **BR-Q-05** "Call next" advances the queue only; it never creates a booking. It succeeds only if `expectedLastCalledNumber` equals the current serving number (compare-and-set).
- **BR-Q-06** The next patient is the WAITING entry with the lowest number. A requeued entry keeps its number and is therefore called next.

- **BR-Q-07** Only a SERVING entry can be completed or skipped; only a WAITING entry can be cancelled; only a SKIPPED entry can be requeued.
- **BR-Q-08** Closing a session with WAITING or SERVING entries requires `force = true`, which cancels WAITING entries ("Session closed") and completes SERVING entries. Sessions dated before today are force-closed nightly at 00:05. CLOSED is terminal.
- **BR-Q-09** Several entries may be SERVING at once in one session (several doctors); the public page shows all of them and the most recently called as "now serving".
- **BR-Q-10** Public responses contain only: session id, label, date, status, current serving number, serving numbers, waiting count, first 50 waiting numbers, last issued number and update time. Adding any field requires a privacy review by the owner.
- **BR-Q-11** The public endpoints expose only sessions dated today or later.
- **BR-Q-12** Caller name, phone and notes of queue entries are removed after `APP_RETENTION_QUEUE_PII_DAYS` (default 90).

7.7 Bookings

- **BR-B-01** Bookings are registered by a doctor or pharmacist on behalf of the caller; patients cannot book themselves in release 1.0.
- **BR-B-02** If an active entry of the same session has the same (normalised) phone number the booking is refused with `409 DUPLICATE_BOOKING` unless staff confirm (`allowDuplicate = true`).
- **BR-B-03** Booking notes are non-clinical (e.g. mobility needs); the UI states this next to the field.
- **BR-B-04** A booking is not linked to a patient record; the link is made optionally when the doctor records the consultation (BR-C-04).

7.8 Status lifecycles (canonical)

Prescription statuses, dispensing statuses and queue statuses are three separate enumerations bound to separate columns; they are never reused for one another.

Prescription status (`prescriptions.status`, enum `PrescriptionStatus`)

From	Action	To / result	Who
(new)	create consultation / add prescription	PENDING	DOCTOR
PENDING	start	IN_PROGRESS	PHARMACIST
PENDING	dispense	DISPENSED	PHARMACIST
IN_PROGRESS	dispense	DISPENSED	PHARMACIST
PENDING, IN_PROGRESS	cancel (reason)	CANCELLED	prescribing DOCTOR, any PHARMACIST
IN_PROGRESS	start (repeat)	IN_PROGRESS (no-op, 200)	PHARMACIST
CANCELLED	cancel (repeat)	CANCELLED (no-op, 200)	as cancel
DISPENSED	start / cancel / dispense	409 INVALID_STATUS_TRANSITION	-
CANCELLED	start / dispense	409 INVALID_STATUS_TRANSITION	-

Dispensing outcome (`dispensings.outcome`, enum `DispensingOutcome`): `FULL` when every item is fully dispensed, otherwise `PARTIAL`. **Dispensed item status** (`dispensing_items.status`, enum `DispensedItemStatus`): `FULL` (= prescribed), `PARTIAL` (0 < qty < prescribed), `NOT_DISPENSED` (0). Both are derived by the server, never sent by the client.

Queue entry status (`queue_entries.status`, enum `QueueEntryStatus` - a separate lifecycle from prescriptions)

From	Action	To / result	Who
(new)	create booking	WAITING	DOCTOR, PHARMACIST
WAITING	call-next / call	SERVING	DOCTOR (session OPEN)
WAITING	cancel (reason)	CANCELLED	DOCTOR, PHARMACIST
SERVING	complete / call-next completeCurrentEntryId	COMPLETED	DOCTOR
SERVING	skip	SKIPPED	DOCTOR
SKIPPED	requeue	WAITING (same number)	DOCTOR, PHARMACIST
WAITING	session close force=true	CANCELLED ("Session closed")	system
SERVING	session close force=true	COMPLETED	system
same as target	repeat call / complete / skip / cancel / requeue	unchanged (no-op, 200)	-
any other	any	409 INVALID_STATUS_TRANSITION	-

Queue session status (`queue_sessions.status`, enum `QueueSessionStatus`)

From	Action	To / result	Who
(new)	create	SCHEDULED	DOCTOR, PHARMACIST
SCHEDULED	open (only on sessionDate)	OPEN	DOCTOR, PHARMACIST
SCHEDULED, OPEN	close (force if active entries)	CLOSED	DOCTOR, PHARMACIST, nightly job
OPEN	open (repeat)	OPEN (no-op)	-
CLOSED	any mutation or booking	409 QUEUE_SESSION_CLOSED	-

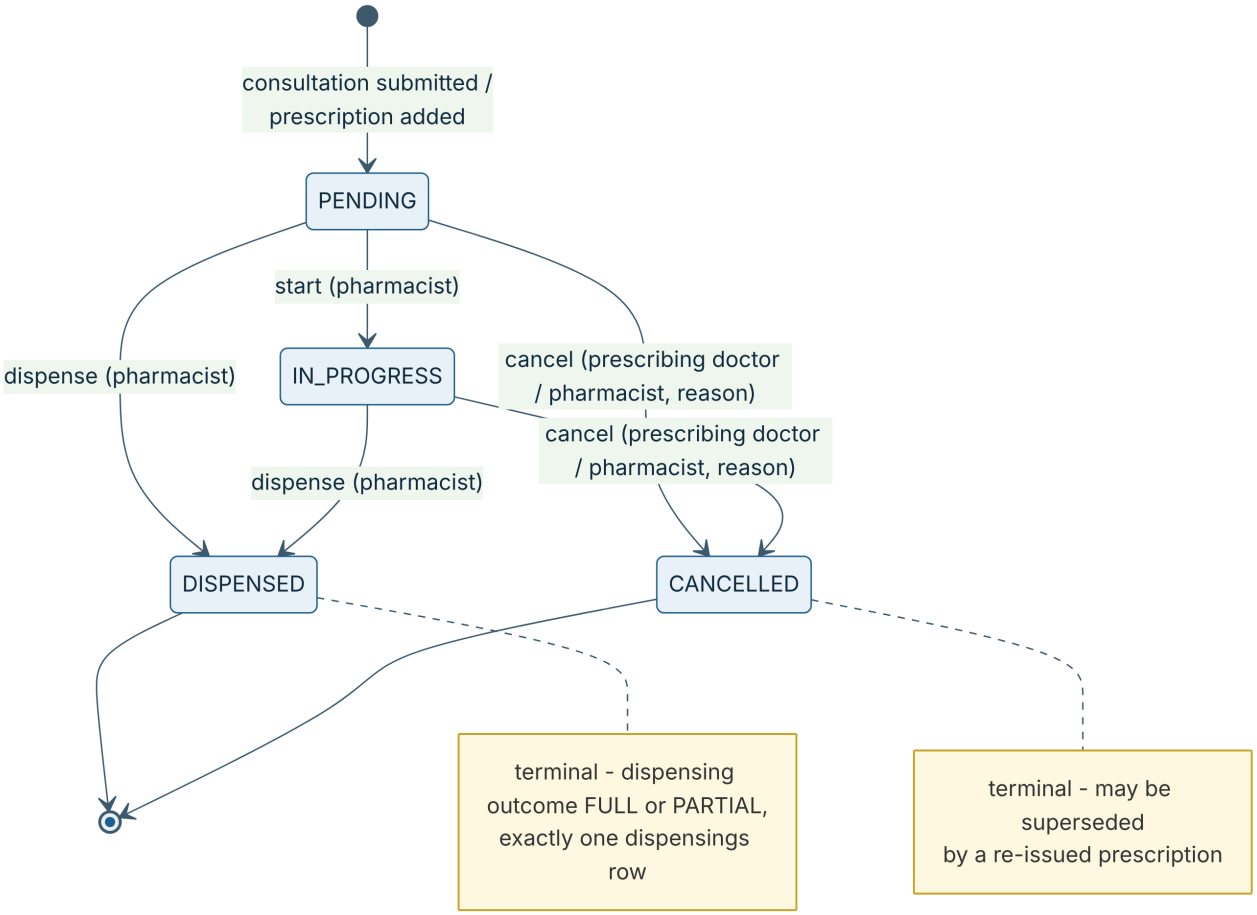


Figure 1. Prescription lifecycle (source: diagrams/prescription-state.mmd)

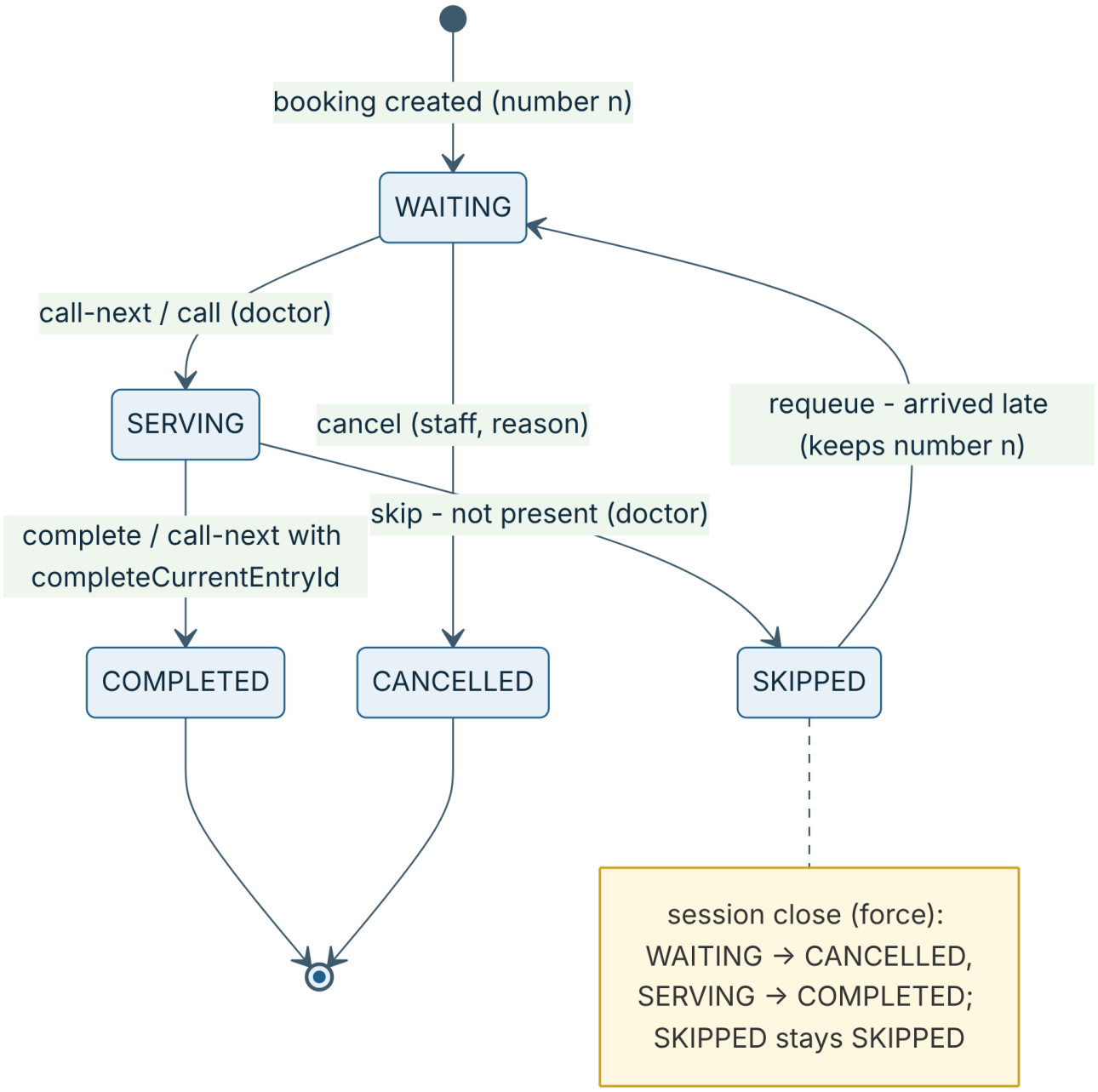


Figure 2. Queue entry lifecycle (source: diagrams/queue-entry-state.mmd)

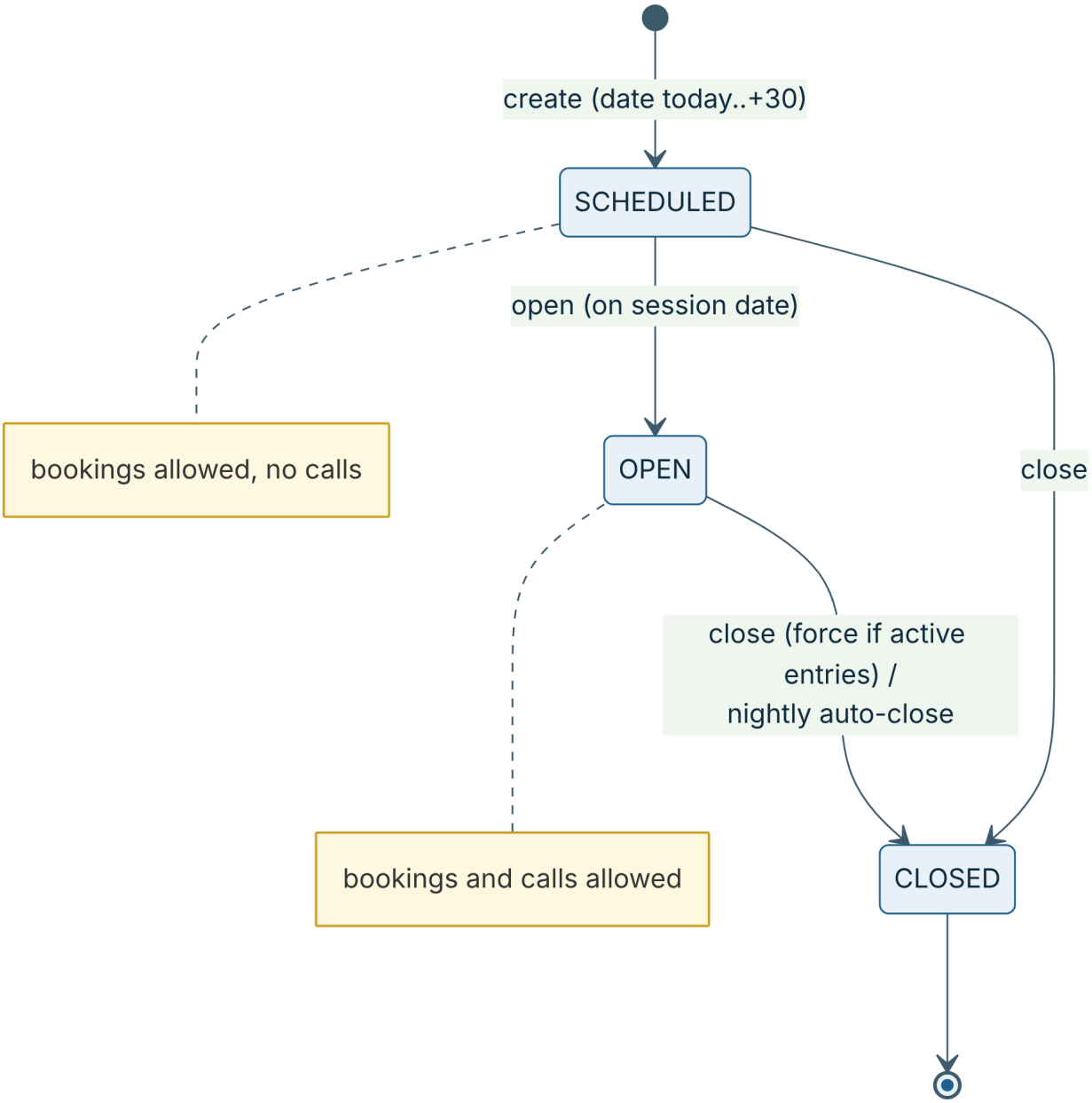


Figure 3. Queue session lifecycle (source: diagrams/queue-session-state.mmd)

8. End-to-end workflows

Sequence diagrams for these workflows are in 02-Database-and-System-Architecture.pdf section 2.5.

8.1 Workflow A - Existing patient consultation

1. Doctor logs in (UC-01) and opens *Patient search*.
2. Doctor enters the patient number; the frontend calls `GET /patients/by-number/{n}`.
3. The backend normalises the number, finds the patient and audits `PATIENT_VIEWED`.
4. The frontend opens the history (`GET /patients/{id}/medical-history`): conditions summary and the 10 most recent consultations with diagnoses, prescriptions and dispensing results.
5. Doctor reviews previous consultations and prescriptions (older pages on demand).
6. Doctor clicks *New consultation* (from the queue console the current queue entry is carried along).
7. Doctor records presenting complaint, clinical notes and 1-10 diagnoses.
8. Doctor adds 1-30 medicines using catalogue autocomplete or free text.
9. Doctor submits; the frontend sends `POST /consultations` with an Idempotency-Key and disables the button.
10. The backend validates everything and persists consultation, diagnoses, prescription (PENDING) and items in one transaction.
11. The prescription appears in the pharmacist queue on the next poll (<= 10 s).
12. The pharmacist opens it, optionally marks it IN_PROGRESS, and confirms quantities per item.
13. The backend records the dispensing (pharmacist, time, per-item quantities) and sets DISPENSED.
14. The doctor's consultation screen, history and *My prescriptions* show "Dispensed" (FULL or PARTIAL) on the next poll.

Failure handling: validation errors are shown at the field (including item rows); a `409/422` leaves the form intact; a network error or timeout keeps the form and a retry reuses the same key so the consultation is never created twice.

8.2 Workflow B - New patient registration and consultation

1. Doctor searches the number; the backend returns `404 PATIENT_NOT_FOUND`.
2. The UI offers *Register new patient* with the number pre-filled and shows possible name matches (BR-PAT-05).
3. Doctor enters name, sex, date of birth or estimated age and optional phone, address and allergies.
4. The backend validates, checks the number for uniqueness and registers the patient (`POST /patients`, own transaction). A duplicate number returns `409 DUPLICATE_PATIENT_NUMBER` with a link to the existing patient.
5. The UI continues directly to the consultation form for the new patient.
6. Steps 7-14 of Workflow A follow.

Partial failure: registration and consultation are intentionally separate transactions. After step 4 the patient exists and is correct; if the consultation then fails, nothing of the consultation is stored, the doctor stays on the form with all entries, and retries. No orphaned prescriptions or half-written consultations can exist (BR-C-05). The alternative - one combined endpoint - was rejected because it would make the registration screen and the consultation form one unrecoverable unit and complicate retries.

8.3 Workflow C - Telephone booking

1. A patient phones the dispensary.
2. A doctor or pharmacist opens *New booking*, selects today's (or a future) session, enters the caller's name, optional phone and a non-clinical note.
3. The backend locks the session row, checks status, limit and duplicate phone, increments `last_issued_number` and inserts the entry in the same transaction (`POST /queue-sessions/{id}/entries`, Idempotency-Key).
4. The number (e.g. 6) is shown in large type; staff tell the caller.
5. The public display shows the new waiting number on its next poll.

Errors: closed session (`409 QUEUE_SESSION_CLOSED`), full session (`409 QUEUE_SESSION_FULL`), duplicate phone (`409 DUPLICATE_BOOKING`, confirm to override), validation (`400`). **Concurrency:** simultaneous bookings serialise on the session row lock; numbers are unique and gap-free (verified by BT-017 with 100 parallel requests).

8.4 Workflow D - Public queue display

1. A visitor opens `/public/queue` (list of today's queues) or a direct link / QR code `/public/queue/{sessionId}`.
2. The page calls `GET /public/queue-sessions/{id}`; the response contains only numbers, counts, status and label.
3. The page shows NOW SERVING in very large digits, the other numbers being served, the waiting numbers and the waiting count.
4. Every 5 s it polls with `If-None-Match`; unchanged queues return `304`. Changes appear within 7 s (NFR-004).
5. A patient may type their own number to see how many are ahead; the number never leaves the browser (FR-045).

8.5 Workflow E - Calling the next patient

1. The doctor presses *Complete & call next* (or *Call next*).
2. The frontend sends `expectedLastCalledNumber` (the serving number on screen) and, if a patient is being served, `completeCurrentEntryId`.
3. The backend locks the session row, compares the expected number, completes the current entry, selects the lowest WAITING entry, marks it SERVING and sets the session's serving number - one transaction.
4. If another request advanced the queue first, the backend returns `409 QUEUE_STATE_CHANGED`; the UI refreshes and shows the new number without advancing again.
5. If nobody is waiting, the response says so (`calledEntry = null`); the completion still applies.
6. The public display shows the new number on its next poll.

Number generation: numbers come from `queue_sessions.last_issued_number`, incremented under a row lock in the booking transaction. They restart at 1 for each queue session (at least daily, BR-Q-03).

9. Use cases

Each use case lists its main flow; alternative flows reference error codes from section 11. Acceptance criteria are those of the referenced FRs.

UC	Name	Actor	Preconditions	Main flow	Alternatives	Postconditions	FRs
UC-01	Log in	Staff	Account enabled	Enter username/ password → dashboard for role	Wrong credentials (401), locked (401 ACCOUNT_LOCKED), must change password -> /change-password	Session established, audit AUTH_LOGIN_SUCCEEDED	FR-001 FR-004
UC-02	Find patient and review history	Doctor	Logged in	Enter patient number → history page	Not found → UC-03; search by name/phone	PATIENT_VIEWED, PATIENT_HISTORY_VIEWED audited	FR-011, FR-016
UC-03	Register patient	Doctor	Number not found	Fill form → save → consultation form	Duplicate number (409) -> open existing; DOB unknown → estimated age	Patient exists	FR-013
UC-04	Record consultation with prescription	Doctor	Patient selected	Complaint, notes, diagnoses, items → submit → confirmation	Inactive medicine (422), validation (400), retry after network error	Consultation + PENDING prescription	FR-017. FR-024
UC-05	Correct a prescription	Doctor	Own consultation < 24 h, prescription not dispensed	Cancel with reason → issue replacement	Already dispensed (409), > 24 h (422)	Old CANCELLED, new PENDING superseding it	FR-030
UC-06	Dispense prescription	Pharmacist	Prescription PENDING/ IN_PROGRESS	Open → start → confirm quantities → Mark as Dispensed	Short supply → partial with notes; nothing available → cancel; already dispensed by colleague (409)	DISPENSED with per-item record	FR-025
UC-07	Register telephone booking	Doctor/ Pharmacist	Session SCHEDULED/ OPEN	Select session, enter caller -> number shown	Duplicate phone → confirm; session closed/ full (409)	WAITING entry with new number	FR-036
UC-08	Run a clinic queue	Doctor	Session OPEN	Call next → consult → complete & call next	No-show → skip; late arrival → requeue; emergency → call specific; 409 QUEUE_STATE_CHANGED → refresh	Entries progress through lifecycle	FR-035 FR-039
UC-09	Watch public queue	Public	Session today	Open public page → numbers update automatically	Connection lost → banner, keeps retrying	-	FR-043

UC	Name	Actor	Preconditions	Main flow	Alternatives	Postconditions	FRs
UC-10	Manage staff accounts	Admin	Logged in as ADMIN	Create user with temporary password, assign roles, disable, reset	Duplicate username (409); self-disable / last admin (422)	Account changes audited	FR-007
UC-11	Maintain medicine catalogue	Pharmacist/ Admin	Logged in	Add, edit, deactivate medicine	Duplicate (409)	Catalogue updated; history unchanged	FR-033
UC-12	Review audit trail	Admin	Logged in as ADMIN	Filter by patient, user, action, period	Range > 366 days (400)	AUDIT_LOG_VIEWED audited	FR-046

10. Input validation rules

Validation runs on the server (Bean Validation and service rules - authoritative) and is mirrored in the frontend with Zod for immediate feedback. All text inputs are trimmed; empty strings in optional fields are converted to `null`. Lengths are characters and equal the database column sizes (verified automatically). The complete per-field list for every request is in 03-REST-API-Documentation.pdf.

Field(s)	Rule	Error
`username` (create user)	`^[A-Za-z0-9._-]{3,50}$`, stored lower-case, unique	400 VALIDATION_FAILED / 409 DUPLICATE_USERNAME
passwords (`newPassword`, `temporaryPassword`)	12-128 chars; not equal to current; must not contain the username; not on deny-list	400 / 422 PASSWORD_POLICY_VIOLATION
`patientNumber`	trim + upper-case, `^[A-Z0-9][A-Z0-9-]{2,19}$`, unique, immutable	400 / 409 DUPLICATE_PATIENT_NUMBER
`fullName` (patient, user), `patientName` (booking)	1-150 chars, not blank	400
`sex`	MALE, FEMALE, OTHER, UNKNOWN	400
`dateOfBirth` / `estimatedAgeYears`	exactly one; DOB between 1900-01-01 and today; age 0-130	400
`phoneNumber`, `contactPhone`	optional; remove spaces, hyphens, parentheses; then `^\+?[0-9]{7,15}$`	400
`address` / `knownAllergies`	<= 300 / <= 1000 chars	400
`presentingComplaint` / `clinicalNotes`	<= 1000 / <= 5000 chars	400
`diagnoses`	1-10 entries; `conditionText` 1-500; `icd10Code` optional `^[A-Z][0-9]{2}(\.[0-9A-Z]{1,4})?$`	400
`prescription.items`	1-30 entries; `medicineId` xor `medicineName` (1-200); `strength` <= 50; `dosageForm` enum; `dose`, `frequency` 1-100; `durationDays` 1-365 or null; `quantity` 0.01-99 999.99, max 2 decimals; `quantityUnit` 1-30; `instructions` <= 500	400; 422 MEDICINE_INACTIVE / REFERENCED_RESOURCE_NOT_FOUND
`notes` (prescription, dispensing)	<= 1000 chars	400
dispense `items`	each prescription item exactly once; `quantityDispensed` 0..prescribed, max 2 decimals; `note` <= 500 required when below prescribed; not all zero	422 DISPENSE_* / NOTHING_DISPENSED
`reason` (cancel prescription / booking)	3-500 chars	400
`reason` (skip)	optional, <= 300 chars	400
`sessionDate`	today..today+30 (APP_TIMEZONE)	422 INVALID_SESSION_DATE
session `name`	1-100 chars, unique per date case-insensitively	400 / 409 DUPLICATE_QUEUE_SESSION
booking `notes`	<= 300 chars, non-clinical (UI hint)	400

Field(s)	Rule	Error
`expectedLastCalledNumber`	required key, integer 1-999 or null	400
`Idempotency-Key` header	`^[A-Za-z0-9_-]{8,100}$` (UUID v4 recommended)	400 IDEMPOTENCY_KEY_REQUIRED
paging `page` / `size`	page >= 0; size 1-100 (medical history 1-50)	400
`search` (patients)	2-100 chars	400
`version` (updates)	integer >= 0, must equal current	409 CONCURRENT_MODIFICATION
unknown JSON properties in requests	rejected	400 MALFORMED_REQUEST

11. Error-handling requirements

- **EH-01** Every error response uses RFC 9457 Problem Details (`application/problem+json`) with `type`, `title`, `status`, `detail`, `instance`, `code`, `requestId`, `timestamp`, and `errors[]` for field problems.
- **EH-02** Clients branch on `code` (stable, catalogued below), never on `detail` text.
- **EH-03** Validation reports all failing fields at once with JSON paths such as `prescription.items[2].quantity`.
- **EH-04** Responses never contain stack traces, SQL, class names or internal identifiers other than resource UUIDs. Unexpected errors return `500 INTERNAL_ERROR` with the `requestId` that appears in the server log.
- **EH-05** Database constraint violations are mapped by constraint name to codes (e.g. `uq_patients_patient_number` → `DUPLICATE_PATIENT_NUMBER`). Unmapped violations are logged as errors and returned as 500.
- **EH-06** Optimistic-lock failures return `409 CONCURRENT_MODIFICATION`; the UI reloads the record and asks the user to re-apply.
- **EH-07** Lock waits longer than 5 s (`lock_timeout`) return `503 SERVICE_UNAVAILABLE`; clients may retry with the same idempotency key.
- **EH-08** The frontend keeps all form input after any error, maps `errors[]` to fields, shows a toast for 409/422 with the `detail`, and an error panel with the request id for 500/503.
- **EH-09** GET queries retry twice with exponential backoff; mutations are never retried automatically, except that a user-initiated retry reuses the same idempotency key.
- **EH-10** On `401 TOKEN_EXPIRED` the frontend refreshes once and retries the request; on refresh failure it clears the cache and redirects to `/login?next=<path>`.
- **EH-11** `404 PATIENT_NOT_FOUND` on number lookup is an expected business outcome, rendered as "not found - register?" rather than an error.
- **EH-12** Exceptions are never swallowed: each is either handled with a defined response or logged once (with request id) by the global handler.
- **EH-13** Workflow B partial failure is handled as described in 8.2; no compensating deletes are ever needed.

11.1 Error code catalogue

Code	HTTP	Title	When it is returned
`MALFORMED_REQUEST`	400	Malformed request	Body is not valid JSON, has wrong JSON types, or an unknown enum value.
`VALIDATION_FAILED`	400	Validation failed	Bean Validation (field or cross-field) failed for the body, path or query parameters. errors[] lists every failing field.
`IDEMPOTENCY_KEY_REQUIRED`	400	Idempotency key required	A POST that requires the Idempotency-Key header was sent without it, or the key does not match ^[A-Za-z0-9_-]{8,100}$.
`UNAUTHENTICATED`	401	Authentication required	No bearer token, or the token is malformed or has an invalid signature.
`TOKEN_EXPIRED`	401	Access token expired	The bearer token is well-formed but expired. Client should call POST /auth/refresh once and retry.
`INVALID_CREDENTIALS`	401	Invalid credentials	Unknown username or wrong password (same response for both).
`ACCOUNT_LOCKED`	401	Account temporarily locked	Correct password but the account is locked after 5 consecutive failures. Returned only when the password is correct, so it cannot be used to enumerate users.
`ACCOUNT_DISABLED`	401	Account disabled	Correct password but the account is disabled.
`REFRESH_TOKEN_INVALID`	401	Session expired	Refresh cookie missing, unknown, expired or revoked. Client must show the login screen.
`REFRESH_TOKEN_ROTATED`	401	Refresh token already rotated	Refresh token was rotated less than 30 s ago (parallel refresh from another tab). The browser already holds the new cookie; client retries refresh once after 500 ms.
`SESSION_REVOKED`	401	Session revoked	A rotated refresh token was reused after the 30 s grace window (possible theft). The whole token family is revoked.
`FORBIDDEN`	403	Access denied	Authenticated user lacks the role required by the endpoint.
`PASSWORD_CHANGE_REQUIRED`	403	Password change required	users.must_change_password is TRUE; only GET /auth/me, POST /auth/change-password and POST /auth/logout are allowed.
`NOT_RESOURCE_OWNER`	403	Not the owner of this resource	Role is sufficient but an ownership rule fails, e.g. a doctor cancels another doctor's prescription.
`RESOURCE_NOT_FOUND`	404	Resource not found	Unknown route or generic missing resource.
`PATIENT_NOT_FOUND`	404	Patient not found	No patient with the given id or patient number. For lookup by number the UI offers registration.
`CONSULTATION_NOT_FOUND`	404	Consultation not found	No consultation with the given id.
`PRESCRIPTION_NOT_FOUND`	404	Prescription not found	No prescription with the given id.
`QUEUE_SESSION_NOT_FOUND`	404	Queue session not found	No queue session with the given id.
`QUEUE_ENTRY_NOT_FOUND`	404	Queue entry not found	No queue entry with the given id.
`USER_NOT_FOUND`	404	User not found	No user with the given id.
`MEDICINE_NOT_FOUND`	404	Medicine not found	No medicine with the given id (path parameter).
`DUPLICATE_PATIENT_NUMBER`	409	Patient number already exists	uq_patients_patient_number violated. errors[0].field = patientNumber; existingPatientId is returned so the UI can open that record.

Code	HTTP	Title	When it is returned
DUPLICATE_USERNAME	409	Username already exists	uq_users_username violated.
DUPLICATE_MEDICINE	409	Medicine already exists	uq_medicines_name_strength_form violated.
DUPLICATE_QUEUE_SESSION	409	Queue session already exists	uq_queue_sessions_date_name violated.
DUPLICATE_BOOKING	409	Possible duplicate booking	An active (WAITING/SERVING) entry in the same session has the same contact phone and allowDuplicate was not true. existingQueueNumber is returned.
CONCURRENT_MODIFICATION	409	Record changed by someone else	Optimistic-lock version in the request does not match (JPA OptimisticLockException). Client reloads and re-applies.
INVALID_STATUS_TRANSITION	409	Invalid status transition	Requested transition is not allowed from the current status (prescription or queue entry). currentStatus is returned.
QUEUE_STATE_CHANGED	409	Queue changed	call-next expectedLastCalledNumber differs from the session's last_called_number (someone else already called). currentServingNumber is returned; UI refreshes.
QUEUE_SESSION_NOT_OPEN	409	Queue session not open	Calling, completing or skipping requires status OPEN.
QUEUE_SESSION_CLOSED	409	Queue session closed	Booking or any mutation on a CLOSED session.
QUEUE_SESSION_FULL	409	Queue session full	last_issued_number = 999.
QUEUE_SESSION_HAS_ACTIVE_ENTRIES	409	Queue session has active entries	close without force=true while WAITING or SERVING entries exist. waitingCount and servingCount are returned.
QUEUE_ENTRY_ALREADY_LINKED	409	Queue entry already linked	uq_consultations_queue_entry violated: another consultation already references the queue entry.
ACTIVE_PRESCRIPTION_EXISTS	409	Active prescription exists	uq_prescriptions_active_per_consultation violated: the consultation already has a non-cancelled prescription.
REFERENCED_RESOURCE_NOT_FOUND	422	Referenced resource not found	An id inside the request body (patientId, queueEntryId, medicineId, assignedDoctorId, prescriptionItemId, supersedesPrescriptionId) does not exist. errors[0].field names it.
MEDICINE_INACTIVE	422	Medicine is not available	A prescription item references a catalogue medicine with active = false.
CONSULTATION_LOCKED	422	Consultation locked	A prescription is added to a consultation more than 24 h after consultation_at. (A different doctor gets 403 NOT_RESOURCE_OWNER.)
INVALID_SUPERSEDED_PRESCRIPTION	422	Invalid superseded prescription	supersedesPrescriptionId is not a CANCELLED prescription of the same consultation, or is already superseded.
QUEUE_ENTRY_NOT_IN_SESSION	422	Queue entry not in session	completeCurrentEntryId (call-next) or queueEntryId (consultation) refers to an entry of a different session or a non-SERVING entry where SERVING is required.
DISPENSE_ITEMS_MISMATCH	422	Dispense items do not match	items[] must contain every prescription item of the prescription exactly once.

Code	HTTP	Title	When it is returned
`DISPENSE_QUANTITY_EXCEEDS_PRESCRIBED`	422	Quantity exceeds prescribed	quantityDispensed > prescribed quantity for an item.
`NOTHING_DISPENSED`	422	Nothing dispensed	Every item has quantityDispensed = 0. Cancel the prescription instead.
`DISPENSE_NOTE_REQUIRED`	422	Note required	An item with quantityDispensed below the prescribed quantity has no note.
`INVALID_SESSION_DATE`	422	Invalid session date	sessionDate is before today or more than 30 days ahead (APP_TIMEZONE).
`ASSIGNED_USER_NOT_DOCTOR`	422	Assigned user is not a doctor	assignedDoctorId refers to a user without the DOCTOR role or a disabled user.
`SELF_MODIFICATION_NOT_ALLOWED`	422	Cannot change own account this way	An admin tries to disable themselves or remove their own ADMIN role.
`LAST_ADMIN_REQUIRED`	422	At least one admin required	The change would leave no enabled user with the ADMIN role.
`CURRENT_PASSWORD_INCORRECT`	422	Current password incorrect	change-password currentPassword is wrong (422, not 401, so the client does not trigger token refresh).
`PASSWORD_POLICY_VIOLATION`	422	Password does not meet policy	New password equals the current password, contains the username, or is on the deny-list.
`IDEMPOTENCY_KEY_REUSED`	422	Idempotency key reused	The same Idempotency-Key was used by the same user for a different operation or a different request body within 24 h.
`RATE_LIMITED`	429	Too many requests	Rate limit exceeded (login 10/min/IP, public queue 120/min/IP). Retry-After header is set.
`INTERNAL_ERROR`	500	Internal server error	Unexpected exception. Details are logged server-side with requestId; nothing internal is returned.
`SERVICE_UNAVAILABLE`	503	Service unavailable	Database unreachable, a row lock could not be obtained within lock_timeout (5 s; client may retry), or the application is shutting down.

12. Security and privacy requirements

12.1 Authentication and sessions

- **SEC-01** Staff authenticate with username and password; passwords are hashed with BCrypt (cost 12) and never logged or returned.
- **SEC-02** Password policy: 12-128 characters, no composition rules, not containing the username, not on a deny-list of common passwords; temporary passwords must be changed at first login.
- **SEC-03** Lockout after 5 consecutive failures for 15 minutes; login rate limit 10/min/IP.
- **SEC-04** Access tokens: HS256 JWT, 15 minutes, held in browser memory only. Refresh tokens: opaque 256-bit random values, stored hashed, rotated on every use, family revoked on reuse, HttpOnly + Secure + SameSite=Strict cookie scoped to `/api/v1/auth`, absolute lifetime 12 h. Client idle logout after 30 minutes.

12.2 Authorisation and least privilege

- **SEC-05** Role checks on every endpoint in the backend (`@PreAuthorize`), derived from `openapi.yaml`; deny by default for unmapped routes.
- **SEC-06** Ownership checks in services (e.g. cancelling own prescriptions only).
- **SEC-07** Pharmacists receive only the data needed to dispense; admins receive no clinical data; the public receives only queue numbers.
- **SEC-08** Denied access to patient-related endpoints is audited as `ACCESS_DENIED`.

12.3 Application security

- **SEC-09** All database access through JPA/parameterised queries; dynamic sorting uses a whitelist; no string-concatenated SQL.
- **SEC-10** Output escaping by React; no `dangerouslySetInnerHTML`; Content-Security-Policy without `unsafe-inline` scripts (nonces) and `frame-ancestors 'none'`.
- **SEC-11** CSRF: API calls use bearer tokens (not cookies); the cookie-authenticated refresh/logout endpoints require SameSite=Strict and the `X-Requested-With` header.
- **SEC-12** HTTPS only in production with HSTS; the database is not reachable from outside the private network.
- **SEC-13** Secrets come only from environment variables or a secret store; the application refuses to start with a missing or short JWT secret; `.env` files are git-ignored; nothing secret uses `NEXT_PUBLIC_`.
- **SEC-14** Request size limit 256 KB; JSON unknown properties rejected; rate limits on public endpoints.
- **SEC-15** Dependencies scanned in CI; no critical/high findings at release.

12.4 Privacy and clinical data

- **SEC-16** Logs contain ids and request ids, never passwords, tokens, clinical notes, diagnoses, medicine lists, names or phone numbers (NFR-010).
- **SEC-17** API responses with clinical data carry `Cache-Control: no-store`; the frontend never writes clinical data to localStorage, sessionStorage or IndexedDB and clears its in-memory cache on logout and idle timeout.
- **SEC-18** Public endpoints return a dedicated DTO; a key-whitelist test guards it (BR-Q-10).
- **SEC-19** Endpoints that need only queue status never return medical history or patient identifiers.

12.5 Backup, retention and deletion

- **SEC-20** Daily encrypted base backups plus continuous WAL archiving to off-site storage, 35-day retention; quarterly restore drill (NFR-006).
- **SEC-21** Proposed retention defaults (owner to confirm, Q-09): clinical records (patients, consultations, prescriptions, dispensings) retained indefinitely in release 1.0; audit events at least 7 years; queue PII 90 days; refresh tokens deleted 7 days after expiry; idempotency keys after 24 h.
- **SEC-22** There is no patient deletion feature in release 1.0. Requests for erasure or correction must be handled by the owner under the applicable law using a documented manual procedure (future feature).

12.6 Legal, clinical and operational items the owner must confirm before production

The DMS is designed with privacy and security as core requirements, but **no claim is made that it complies with any specific law or regulation**. Before production use the owner must confirm with qualified advisers:

1. Which data-protection law applies and what it requires (for a dispensary in Sri Lanka, for example, the Personal Data Protection Act No. 9 of 2022 may apply), including lawful basis, notices to patients, data-subject rights and breach notification.
2. Medical-record retention periods and whether clinical records may ever be deleted or must be amended only by addenda.
3. Prescription and dispensing record-keeping rules for the pharmacy (who may dispense, what must be recorded, how long).
4. Whether pharmacists should see diagnoses (Q-02) and whether dual-role staff may dispense their own prescriptions (Q-04).
5. Professional registration numbers to be recorded for doctors and pharmacists.
6. Hosting location, backup storage location and access controls for the server.
7. Operational procedures: account provisioning/deprovisioning, password reset identity checks, incident response, restore testing, and what staff do when the system is unavailable (paper fallback).
8. Wording of any notice shown on the public queue page and in the waiting area.

13. Audit logging requirements

- **AUD-01** Audit events are written for authentication events, every clinical, pharmacy, queue and administrative write, and every read of patient demographics, medical histories, consultations and prescription details.
- **AUD-02** List endpoints polled by dashboards (prescription list, queue status) are not audited per call; opening a record is.
- **AUD-03** Each event stores time, actor id and username, action, entity type and id, patient id (when patient data is touched), outcome (SUCCESS, FAILURE, DENIED), request id, client IP and user agent.
- **AUD-04** `details` contains only the keys listed below - never passwords, tokens, clinical notes, diagnoses, reasons or medicine names.
- **AUD-05** Write events are inserted in the same transaction as the change; read events are inserted in the request's read transaction (if auditing fails the request fails - no silent loss).
- **AUD-06** `audit_events` is append-only (database trigger); only ADMIN can read it, and reading it is audited.
- **AUD-07** The application records what the doctor prescribed (immutable items), what the pharmacist dispensed (dispensing items), who did each and when - independent of the audit log.

13.1 Audit action catalogue

Action	Entity type	Sets patient_id	Written when	Allowed details keys
`AUTH_LOGIN_SUCCEEDED`	SESSION	no	POST /auth/login 200	familyId
`AUTH_LOGIN_FAILED`	SESSION	no	POST /auth/login 401 (any reason)	reason (INVALID_CREDENTIALS|ACCOUNT_LOCKED|ACCOUNT_DISABLED)
`AUTH_ACCOUNT_LOCKED`	USER	no	5th consecutive failure locks the account	lockedUntil
`AUTH_LOGOUT`	SESSION	no	POST /auth/logout	familyId
`AUTH_REFRESH_REUSE_DETECTED`	SESSION	no	Rotated refresh token reused after grace window; family revoked	familyId
`AUTH_PASSWORD_CHANGED`	USER	no	POST /auth/change-password 204	-
`ACCESS_DENIED`	AUDIT	yes	403 on any endpoint that touches patient or prescription data (outcome DENIED)	method, path, requiredRoles
`USER_CREATED`	USER	no	POST /users	username, roles
`USER_UPDATED`	USER	no	PUT /users/{id}	changedFields, roles, enabled
`USER_PASSWORD_RESET`	USER	no	POST /users/{id}/reset-password	-
`PATIENT_CREATED`	PATIENT	yes	POST /patients 201	patientNumber
`PATIENT_UPDATED`	PATIENT	yes	PUT /patients/{id} 200	changedFields (names only, no values)
`PATIENT_VIEWED`	PATIENT	yes	GET /patients/{id}, GET /patients/by-number/{n} 200	via (ID|NUMBER)
`PATIENT_SEARCHED`	PATIENT	no	GET /patients?search=	search, resultCount
`PATIENT_HISTORY_VIEWED`	PATIENT	yes	GET /patients/{id}/medical-history 200	page, size
`CONSULTATION_CREATED`	CONSULTATION	yes	POST /consultations 201	diagnosisCount, prescriptionId, itemCount, queueEntryId
`CONSULTATION_VIEWED`	CONSULTATION	yes	GET /consultations/{id} 200	-
`PRESCRIPTION_CREATED`	PRESCRIPTION	yes	Prescription created with a consultation or via POST /consultations/{id}/prescriptions	prescriptionNumber, itemCount, supersedesPrescriptionId
`PRESCRIPTION_VIEWED`	PRESCRIPTION	yes	GET /prescriptions/{id} 200	-
`PRESCRIPTION_STARTED`	PRESCRIPTION	yes	POST /prescriptions/{id}/start (status changed)	fromStatus, toStatus

Action	Entity type	Sets patient_id	Written when	Allowed details keys
`PRESCRIPTION_DISPENSED`	PRESCRIPTION	yes	POST /prescriptions/{id}/dispense 200	fromStatus, toStatus, outcome, dispensingId
`PRESCRIPTION_CANCELLED`	PRESCRIPTION	yes	POST /prescriptions/{id}/cancel 200	fromStatus, toStatus (reason text is NOT copied)
`MEDICINE_CREATED`	MEDICINE	no	POST /medicines 201	name, strength, dosageForm
`MEDICINE_UPDATED`	MEDICINE	no	PUT /medicines/{id} 200	changedFields, active
`QUEUE_SESSION_CREATED`	QUEUE_SESSION	no	POST /queue-sessions 201	sessionDate, name
`QUEUE_SESSION_OPENED`	QUEUE_SESSION	no	POST /queue-sessions/{id}/open	-
`QUEUE_SESSION_CLOSED`	QUEUE_SESSION	no	POST /queue-sessions/{id}/close or nightly auto-close	force, cancelledCount, completedCount, automatic
`BOOKING_CREATED`	QUEUE_ENTRY	no	POST /queue-sessions/{id}/entries 201	queueNumber, channel, duplicateOverride
`BOOKING_CANCELLED`	QUEUE_ENTRY	no	POST /queue-entries/{id}/cancel	queueNumber, fromStatus
`QUEUE_ENTRY_CALLED`	QUEUE_ENTRY	no	call-next or POST /queue-entries/{id}/call	queueNumber, fromStatus, viaCallNext
`QUEUE_ENTRY_COMPLETED`	QUEUE_ENTRY	no	POST /queue-entries/{id}/complete or call-next completeCurrentEntryId	queueNumber
`QUEUE_ENTRY_SKIPPED`	QUEUE_ENTRY	no	POST /queue-entries/{id}/skip	queueNumber
`QUEUE_ENTRY_REQUEUED`	QUEUE_ENTRY	no	POST /queue-entries/{id}/requeue	queueNumber, requeueCount
`AUDIT_LOG_VIEWED`	AUDIT	no	GET /audit-events	filters
`RETENTION_JOB_EXECUTED`	AUDIT	no	Nightly retention job	anonymizedQueueEntries, deletedRefreshTokens, deletedIdempotencyKeys, autoClosedSessions

14. Data and reporting

Release 1.0 provides operational screens only (no reports module). Owners can obtain counts with SQL (examples in 02-Database-and-System-Architecture.pdf section 6). Data ownership: all data belongs to the dispensary; the system provides no export in release 1.0 beyond database backups.

15. Assumptions, constraints and unresolved questions

15.1 Assumptions

ID	Assumption
A-01	One dispensary site; one database; staff work on trusted devices on the clinic network or over HTTPS.
A-02	Each patient has (or is given) a clinic-card number that staff can type.
A-03	Doctors may view any patient's history (shared care in one practice); access is audited rather than restricted per doctor.
A-04	User interface language is English in release 1.0.
A-05	Scale: up to 20 staff users, 300 bookings and 200 consultations per day, 50 000 patients over the system's life.
A-06	Dispensary time zone is Asia/Colombo (configurable with APP_TIMEZONE).
A-07	Bookings are made only by staff; patients have no accounts.
A-08	A prescription is dispensed in one visit; leftovers after partial dispensing are not collected later through the system.
A-09	Waiting-room screens are ordinary browsers (TV, tablet or PC) with network access to the server.
A-10	No integration with external systems (laboratory, national ID, insurance, SMS) is needed in release 1.0.

15.2 Constraints

ID	Constraint
C-01	Technology stack as in section 2.6; modular monolith; PostgreSQL with Flyway; Docker Compose for local development.
C-02	No inventory functionality in release 1.0.
C-03	No automated clinical decision support (BR-G-03).
C-04	The public queue page must not expose personal or clinical data.
C-05	All configuration and secrets through environment variables.
C-06	The API contract is `openapi.yaml`; implementations must not add or change fields without updating it.

15.3 Unresolved questions (with proposed defaults used in this release)

ID	Question	Proposed default (implemented)	Impact if changed
Q-01	What is the patient identifier? Clinic card number, national ID, or system-generated?	Staff-entered clinic-card number (D-03).	Pattern and uniqueness rules; possibly a generated number endpoint.
Q-02	Should pharmacists see diagnoses?	No.	Add diagnoses to PrescriptionResponse (privacy review).
Q-03	May pharmacists call/complete/skip patients, i.e. run a doctor's queue?	No; they manage sessions and bookings and requeue late patients.	Add PHARMACIST to x-roles of call operations.
Q-04	May a doctor who is also a pharmacist dispense their own prescription?	Yes, audited.	Add a server check (403) on dispense.
Q-05	Must partially dispensed medicines be collectable later?	No; PARTIAL is final.	New "owed items" model and a second dispensing per prescription.
Q-06	Where does a requeued (late) patient go?	Keeps the original number; called next.	Alternative: issue a new number at the end.
Q-07	How far ahead can telephone bookings be made?	30 days.	Validation constant only.
Q-08	Maximum bookings per session?	999 (3-digit display).	Constraint and display width.
Q-09	Retention periods for clinical records, audit events and queue PII?	Indefinite / 7 years / 90 days.	Retention job configuration.
Q-10	Patient self-booking and SMS notifications?	Not in release 1.0.	New public write endpoint with abuse protection; SMS provider.
Q-11	Should consultations allow addenda/corrections?	No; corrections via new consultation.	New addendum entity.
Q-12	Is a separate RECEPTIONIST role needed?	No; pharmacists handle bookings.	New role in Role enum and x-roles.
Q-13	How are mistyped patient numbers corrected?	Manual, audited database procedure by the administrator.	New admin endpoint.
Q-14	Are printed prescriptions, labels or tickets required?	No.	Print views / PDF generation.
Q-15	Do several doctors share one queue or have separate queues?	Separate sessions per room recommended; sharing supported.	None (both supported).
Q-16	Which laws and professional regulations apply?	Owner to confirm (12.6).	Possibly consent, notices, retention, export features.
Q-17	Is 30 minutes idle logout acceptable at the counters?	Yes.	Environment variable only.
Q-18	Should the public page show the doctor or room name?	Session label only (owner chooses the label text).	None.
Q-19	Is "today minus N years" acceptable for estimated DOB?	Yes.	Alternative: 1 July of the estimated year.

16. Future enhancements

Not part of release 1.0; the architecture leaves room for each (modular packages, versioned API, Flyway migrations).

Enhancement	Notes
Medicine inventory and stock	Stock levels, batches, expiry, goods received, automatic deduction on dispensing, low-stock alerts; would let the editor flag unavailable medicines.
Owed items / partial collection	Track remaining quantities after a PARTIAL dispensing and allow a later dispensing.
Patient portal and self-booking	Patient accounts, OTP login, online booking with abuse protection.
Notifications	SMS/WhatsApp "your number is approaching", booking confirmations.
Server-Sent Events	Push queue changes instead of polling; same DTOs; needs LISTEN/NOTIFY or a broker when scaled out.
Printing	Prescription print-out, dispensing labels, queue tickets.
Clinical extensions	Vitals, structured allergies, ICD-10 catalogue, consultation addenda, referrals, lab results.
Clinical decision support	Interaction/allergy checks only with clinically validated data and owner/clinical-governance approval.
Reporting	Daily activity, dispensing statistics, queue waiting times, CSV export.
Administration	Receptionist role, MFA for staff, SSO, patient merge, patient-number correction endpoint, data export for data-subject requests.
Localisation	Sinhala and Tamil UI.
Multi-branch	Branch dimension on sessions, users and patients.
Billing	Consultation and medicine charges, receipts.

17. Traceability matrix

Functional requirements mapped to API endpoints, database entities, frontend screens and tests. Generated from the same catalogues that drive `openapi.yaml` and the implementation guides, and checked automatically (every operation is traced to a requirement and used by a screen; every test covers a requirement).

Requirement	API endpoints (under /api/v1)	Database entities	Frontend screens	Tests
FR-001 Staff login	POST /auth/login	users, user_roles, refresh_tokens, audit_events	/login	BT-001, FT-001
FR-002 Session refresh with rotation	POST /auth/refresh	refresh_tokens, users, audit_events	/login	BT-002, FT-002
FR-003 Logout	POST /auth/logout	refresh_tokens, audit_events	/change-password	BT-002, FT-015
FR-004 Password change and forced change	POST /auth/change-password GET /auth/me	users, refresh_tokens, audit_events	/change-password	BT-004
FR-005 Account lockout	POST /auth/login	users, audit_events	/login	BT-001
FR-006 Role-based access control	GET /auth/me	user_roles, audit_events	/login	BT-003, BT-035, FT-003
FR-007 Staff account administration	GET /users POST /users GET /users/{userId} PUT /users/{userId} POST /users/{userId}/reset-password	users, user_roles, refresh_tokens, audit_events	/admin/users /admin/users/[userId]	BT-032
FR-008 Staff directory	GET /staff	users, user_roles	/queue /admin/audit-events	BT-003
FR-009 Bootstrap administrator	-	users, user_roles	-	BT-032
FR-010 Idle logout on shared workstations	POST /auth/logout	-	/doctor/dashboard /pharmacist/prescriptions	FT-015
FR-011 Look up patient by patient number	GET /patients/by-number/{patientNumber}	patients, audit_events	/doctor/patients/search /doctor/dashboard	BT-005, FT-004
FR-012 Search patients	GET /patients	patients, audit_events	/doctor/patients/search	FT-004, BT-026
FR-013 Register new patient	POST /patients	patients, idempotency_keys, audit_events	/doctor/patients/new	BT-005, BT-006, BT-010, FT-005
FR-014 Unknown date of birth	POST /patients PUT /patients/{patientId}	patients	/doctor/patients/new /doctor/patients/[patientId]	BT-005, BT-033, FT-005

Requirement	API endpoints (under /api/v1)	Database entities	Frontend screens	Tests
FR-015 Update patient demographics	PUT /patients/{patientId} GET /patients/{patientId}	patients, audit_events	/doctor/patients/[patientId]	BT-007
FR-016 View medical history	GET /patients/{patientId}/medical-history	patients, consultations, consultation_diagnoses, prescriptions, prescription_items, dispensings, dispensing_items, audit_events	/doctor/patients/[patientId]	BT-008, FT-008
FR-017 Record a consultation	POST /consultations	consultations, consultation_diagnoses, audit_events	/doctor/consultations/new	BT-009, FT-006
FR-018 Multi-medicine prescription	POST /consultations	prescriptions, prescription_items	/doctor/consultations/new	BT-011, FT-006
FR-019 Catalogue and free-text medicines	POST /consultations GET /medicines	medicines, prescription_items	/doctor/consultations/new	BT-011, FT-006
FR-020 Atomic consultation submission	POST /consultations	consultations, consultation_diagnoses, prescriptions, prescription_items, idempotency_keys	/doctor/consultations/new	BT-009, BT-010, BT-027, FT-007, FT-016
FR-021 Automatic routing to the pharmacist queue	POST /consultations GET /prescriptions	prescriptions	/doctor/consultations/new /pharmacist/prescriptions	BT-009, FT-009, FT-017
FR-022 Submission confirmation and consultation detail	GET /consultations/{consultationId}	consultations, prescriptions	/doctor/consultations/[consultationId]	FT-008
FR-023 Immutable clinical history	-	consultations, consultation_diagnoses, prescription_items	-	BT-029
FR-024 Link consultation to queue entry	POST /consultations GET /queue-entries/{queueEntryId}	consultations, queue_entries	/doctor/consultations/new	BT-009
FR-025 Pharmacist prescription queue	GET /prescriptions	prescriptions, patients, consultations, users	/pharmacist/prescriptions /pharmacist/dashboard	FT-009, BT-026
FR-026 Prescription detail for dispensing	GET /prescriptions/{prescriptionId}	prescriptions, prescription_items, patients, audit_events	/pharmacist/prescriptions/[prescriptionId]	BT-035
FR-027 Mark in progress	POST /prescriptions/{prescriptionId}/start	prescriptions, audit_events	/pharmacist/prescriptions/[prescriptionId]	BT-012
FR-028 Dispense with per-item record	POST /prescriptions/{prescriptionId}/dispense	dispensings, dispensing_items, prescriptions, audit_events	/pharmacist/prescriptions/[prescriptionId]	BT-014, FT-010
FR-029 Exactly-once dispensing	POST /prescriptions/{prescriptionId}/dispense	dispensings, prescriptions, idempotency_keys	/pharmacist/prescriptions/[prescriptionId]	BT-013, BT-010
FR-030 Cancel prescription	POST /prescriptions/{prescriptionId}/cancel	prescriptions, audit_events	/doctor/consultations/[consultationId] /pharmacist/prescriptions/[prescriptionId]	BT-015, BT-012
FR-031 Add or re-issue a prescription	POST /consultations/{consultationId}/prescriptions	prescriptions, prescription_items	/doctor/consultations/[consultationId]	BT-016

Requirement	API endpoints (under /api/v1)	Database entities	Frontend screens	Tests
FR-032 Doctor sees dispensing status	GET /prescriptions GET /consultations/{consultationId} GET /patients/{patientId}/medical-history	prescriptions, dispensings	/doctor/prescriptions /doctor/consultations/[consultationId] /doctor/patients/[patientId]	FT-008, BT-003
FR-033 Medicine catalogue management	GET /medicines POST /medicines GET /medicines/{medicineId} PUT /medicines/{medicineId}	medicines, audit_events	/medicines	BT-034
FR-034 Create queue session	POST /queue-sessions GET /queue-sessions GET /staff	queue_sessions, audit_events	/queue	BT-021
FR-035 Open and close session	POST /queue-sessions/{queueSessionId}/open POST /queue-sessions/{queueSessionId}/close	queue_sessions, queue_entries, audit_events	/queue /queue/[queueSessionId]	BT-021
FR-036 Telephone and walk-in booking	POST /queue-sessions/{queueSessionId}/entries	queue_sessions, queue_entries, idempotency_keys, audit_events	/bookings/new /queue/[queueSessionId]	BT-017, BT-010, FT-011
FR-037 Duplicate booking warning	POST /queue-sessions/{queueSessionId}/entries	queue_entries	/bookings/new	BT-018, FT-011
FR-038 Cancel booking	POST /queue-entries/{queueEntryId}/cancel	queue_entries, audit_events	/queue/[queueSessionId]	BT-020
FR-039 Call next patient (doctor's queue button)	POST /queue-sessions/{queueSessionId}/call-next	queue_sessions, queue_entries, audit_events	/doctor/dashboard /queue/[queueSessionId]	BT-019, FT-012
FR-040 Call a specific patient	POST /queue-entries/{queueEntryId}/call	queue_entries, queue_sessions	/queue/[queueSessionId]	BT-020
FR-041 Complete, skip and requeue	POST /queue-entries/{queueEntryId}/complete POST /queue-entries/{queueEntryId}/skip POST /queue-entries/{queueEntryId}/requeue	queue_entries, audit_events	/queue/[queueSessionId] /doctor/dashboard	BT-020
FR-042 Staff queue status	GET /queue-sessions/{queueSessionId} GET /queue-sessions/{queueSessionId}/entries	queue_sessions, queue_entries	/doctor/dashboard /queue/[queueSessionId]	FT-012, BT-023

Requirement	API endpoints (under /api/v1)	Database entities	Frontend screens	Tests
FR-043 Public list of today's queues	GET /public/queue-sessions	queue_sessions, queue_entries	/public/queue	BT-022
FR-044 Public queue display	GET /public/queue-sessions/{queueSessionId}	queue_sessions, queue_entries	/public/queue/[queueSessionId]	BT-022, BT-023, FT-013
FR-045 Position helper for patients	GET /public/queue-sessions/{queueSessionId}	-	/public/queue/[queueSessionId]	FT-013
FR-046 Audit trail	-	audit_events	-	BT-024
FR-047 Audit log viewer	GET /audit-events	audit_events	/admin/audit-events	BT-024
FR-048 Idempotent create operations	POST /patients POST /consultations POST /consultations/{consultationId}/prescriptions POST /queue-sessions/{queueSessionId}/entries POST /prescriptions/{prescriptionId}/dispense	idempotency_keys	/doctor/patients/new /doctor/consultations/new /doctor/consultations/[consultationId] /bookings/new /pharmacist/prescriptions/[prescriptionId]	BT-010, FT-007
FR-049 Health checks	GET /actuator/health GET /actuator/health/liveness GET /actuator/health/readiness	-	-	BT-029, FT-017
FR-050 Retention and housekeeping jobs	-	queue_entries, queue_sessions, refresh_tokens, idempotency_keys, audit_events	-	BT-031, BT-021

Appendix A. Test catalogue

A.1 Backend tests

ID	Type	Class / file	What it proves	Covers
BT-001	unit	`auth.AuthServiceTest`	Login success, unknown user, wrong password, lockout after 5 failures, ACCOUNT_LOCKED/ACCOUNT_DISABLED only with correct password, counter reset on success	FR-001, FR-005, NFR-008
BT-002	integration	`auth.RefreshTokenFlowIT`	Refresh rotation; reuse within 30 s → REFRESH_TOKEN_ROTATED without revocation; reuse after 30 s → SESSION_REVOKED and family revoked; logout revokes family; X-Requested-With required	FR-002, FR-003, NFR-009
BT-003	security	`security.EndpointAuthorizationMatrixIT`	Parameterised from openapi.yaml x-roles: every operation x {anonymous, ADMIN, DOCTOR, PHARMACIST} → 401/403/allowed	FR-006, FR-008, FR-032, NFR-012

ID	Type	Class / file	What it proves	Covers
BT-004	security	`security.PasswordChangeRequiredIT`	mustChangePassword user gets 403 PASSWORD_CHANGE_REQUIRED except auth/me, change-password, logout; change-password policy checks and revocation of other families	FR-004
BT-005	controller	`patient.PatientControllerTest`	Registration validation: patient number normalisation/pattern, DOB xor estimatedAgeYears, future DOB, phone normalisation, field lengths → 400 VALIDATION_FAILED with errors[]	FR-011, FR-013, FR-014, NFR-013
BT-006	integration	`patient.PatientRegistrationIT`	Duplicate patient number → 409 DUPLICATE_PATIENT_NUMBER with existingPatientId, including two concurrent registrations of the same number (exactly one 201)	FR-013
BT-007	integration	`patient.PatientUpdateIT`	Optimistic locking: stale version → 409 CONCURRENT_MODIFICATION; patientNumber immutable; estimated DOB preserved	FR-015
BT-008	integration	`patient.MedicalHistoryIT`	History ordering (newest first), paging, cancelled prescriptions included, conditions summary grouping; Hibernate statistics prove <= 8 SQL statements per page (no N+1)	FR-016
BT-009	integration	`consultation.ConsultationCreationIT`	Atomic create: failure in any item (e.g. inactive medicine at item 3) leaves no consultation, diagnosis, prescription or item rows; prescription appears as PENDING	FR-017, FR-020, FR-021, FR-024, NFR-017
BT-010	integration	`common.IdempotencyIT`	Same key + same body → replay (same id, same status, no second row); same key + different body → 422 IDEMPOTENCY_KEY_REUSED; 10 parallel identical requests → 1 row; missing header → 400	FR-013, FR-020, FR-029, FR-036, FR-048
BT-011	controller	`consultation.ConsultationControllerTest`	Item rules: medicineId with medicineName → 400; neither → 400; inactive → 422 MEDICINE_INACTIVE; unknown medicineId/patientId → 422 REFERENCED_RESOURCE_NOT_FOUND; 1-10 diagnoses, 1-30 items, quantity scale	FR-018, FR-019, NFR-013
BT-012	unit	`prescription.PrescriptionStateMachineTest`	Exhaustive transition table for PENDING/IN_PROGRESS/DISPENSED/CANCELLED x {start, dispense, cancel}; same-state no-ops; terminal states never change	FR-027, FR-030
BT-013	concurrency	`pharmacy.DispenseConcurrencyIT`	10 threads dispense the same prescription with different keys → exactly one dispensing row, others 409 INVALID_STATUS_TRANSITION; cancel vs dispense race → one wins	FR-029, NFR-017, NFR-018
BT-014	unit	`pharmacy.DispenseServiceTest`	Item set mismatch, quantity > prescribed, all zero (NOTHING_DISPENSED), missing note, derived item status and outcome FULL/PARTIAL, BigDecimal scale handling	FR-028
BT-015	integration	`prescription.PrescriptionCancelIT`	Doctor can cancel only own prescriptions (403 NOT_RESOURCE_OWNER); pharmacist any; reason required; DISPENSED → 409	FR-030
BT-016	integration	`prescription.AddPrescriptionIT`	Re-issue: active exists → 409; > 24 h → 422 CONSULTATION_LOCKED; supersedes must be cancelled & same consultation & not already superseded; supersededByPrescriptionId populated	FR-031
BT-017	concurrency	`queue.BookingNumberConcurrencyIT`	100 concurrent bookings → numbers 1..100 unique and gap-free; failed transactions leave no gaps; 999 limit → QUEUE_SESSION_FULL	FR-036, NFR-017, NFR-018
BT-018	integration	`queue.DuplicateBookingIT`	Same normalised phone in active entry → 409 DUPLICATE_BOOKING with existingQueueNumber; allowDuplicate=true -> 201; completed/cancelled entries do not count	FR-037

ID	Type	Class / file	What it proves	Covers
BT-019	concurrency	`queue.CallNextConcurrencyIT`	Two concurrent call-next with the same expectedLastCalledNumber → one 200, one 409 QUEUE_STATE_CHANGED; no WAITING entry skipped; completeCurrentEntryId semantics; empty queue → calledEntry null	FR-039, NFR-018
BT-020	unit	`queue.QueueEntryStateMachineTest`	Exhaustive queue-entry transition table (call, complete, skip, requeue, cancel) incl. same-state no-ops and session status preconditions	FR-038, FR-040, FR-041
BT-021	integration	`queue.QueueSessionLifecycleIT`	Create date window, unique name per date, open only today, close without force → 409 with counts, force cancels WAITING and completes SERVING, nightly auto-close	FR-034, FR-035, FR-050
BT-022	security	`queue.PublicQueuePrivacyIT`	Public responses contain exactly the documented keys (JSON key whitelist); seeded names/phones/patient numbers never appear in the body; past sessions → 404; no auth required	FR-043, FR-044, NFR-011
BT-023	controller	`queue.PublicQueueEtagTest`	ETag changes on every queue mutation; If-None-Match → 304 with empty body; Cache-Control no-cache	FR-042, FR-044, NFR-003
BT-024	integration	`audit.AuditTrailIT`	Each action in the AuditAction catalogue is written with correct entity/patient ids and outcome; details contain no clinical text; UPDATE/DELETE on audit_events rejected by trigger	FR-046, FR-047, NFR-026
BT-025	repository	`db.FlywayMigrationIT`	Clean PostgreSQL 16 Testcontainer: all migrations apply; hibernate ddl-auto=validate passes; seed migration only in dev profile	NFR-020
BT-026	repository	`queue.QueueEntryRepositoryIT`	Repository queries: next waiting entry, counts by status, active phone lookup, history batch loaders, prescription list filters	FR-012, FR-025
BT-027	controller	`common.ProblemDetailsTest`	Every error is application/problem+json with type, title, status, code, requestId, timestamp; unknown route → 404 RESOURCE_NOT_FOUND; unexpected exception → 500 without stack trace	FR-020, NFR-019
BT-028	integration	`common.LogRedactionIT`	Captured logs during login, consultation and dispense flows contain no password, token, Authorization header, clinical notes or diagnosis text	NFR-008, NFR-010
BT-029	contract	`contract.OpenApiContractIT`	Generated /v3/api-docs has exactly the paths, methods and response status codes of docs/openapi.yaml; sample responses validate against openapi.yaml schemas	FR-023, FR-049, NFR-022
BT-030	integration	`security.RateLimitIT`	Login > APP_RATE_LIMIT_LOGIN_PER_MINUTE and public > APP_RATE_LIMIT_PUBLIC_PER_MINUTE → 429 RATE_LIMITED with Retry-After	NFR-025
BT-031	integration	`retention.RetentionJobIT`	Queue PII anonymised after APP_RETENTION_QUEUE_PII_DAYS; expired refresh tokens (> 7 days) and idempotency keys deleted; audit row written	FR-050
BT-032	integration	`user.UserAdministrationIT`	Create/update/disable/reset; SELF_MODIFICATION_NOT_ALLOWED; LAST_ADMIN_REQUIRED; disabling revokes refresh tokens; bootstrap admin created only when users table is empty	FR-007, FR-009
BT-033	unit	`common.BusinessClockTest`	Today, session-date window, age and estimated DOB computed in APP_TIMEZONE using an injected Clock (midnight boundary cases)	FR-014, NFR-024
BT-034	integration	`medicine.MedicineCatalogIT`	Create/update/deactivate; case-insensitive uniqueness → 409 DUPLICATE_MEDICINE; search; deactivation does not affect existing prescriptions	FR-033

ID	Type	Class / file	What it proves	Covers
BT-035	security	`security.MedicalDataExposureIT`	PHARMACIST cannot call medical-history, patient or consultation endpoints (403 + ACCESS_DENIED audit); prescription detail contains no diagnoses or clinical notes	FR-006, FR-026, NFR-012

A.2 Frontend tests

ID	Type	File	What it proves	Covers
FT-001	component	`features/auth/LoginForm.test.tsx`	Login form validation, error messages per code (INVALID_CREDENTIALS, ACCOUNT_LOCKED, RATE_LIMITED), redirect by role, mustChangePassword → /change-password	FR-001
FT-002	integration	`lib/api/client.test.ts`	Access token held in memory only; single-flight refresh; 401 TOKEN_EXPIRED → one refresh + retry; REFRESH_TOKEN_ROTATED → retry after 500 ms; refresh failure → /login	FR-002
FT-003	integration	`app/(staff)/layout.test.tsx`	Route guard: anonymous → /login?next=; wrong role → /forbidden; protected content not rendered before session bootstrap	FR-006
FT-004	component	`features/patients/PatientSearch.test.tsx`	Exact number lookup: found → navigate to history; 404 → 'Register new patient' with number pre-filled; name search results list	FR-011, FR-012
FT-005	component	`features/patients/PatientForm.test.tsx`	Zod schema mirrors API: DOB xor estimated age toggle, future DOB, phone pattern, lengths; 409 DUPLICATE_PATIENT_NUMBER shows link to existing patient	FR-013, FR-014
FT-006	component	`features/consultations/ConsultationForm.test.tsx`	Diagnoses 1-10, prescription editor add/remove/reorder lines up to 30, catalogue vs free-text medicine, inactive medicines not offered, quantity 2 decimals	FR-017, FR-018, FR-019
FT-007	integration	`features/consultations/submitConsultation.test.tsx`	Submit button disabled while pending; Idempotency-Key stable across retries of one submission and new after success; network error keeps form data; 422 errors mapped to item fields	FR-020, FR-048
FT-008	component	`features/history/MedicalHistory.test.tsx`	History renders conditions, consultations newest first, status badges (PENDING/IN_PROGRESS/DISPENSED/CANCELLED, FULL/PARTIAL), pagination, empty state	FR-016, FR-022, FR-032
FT-009	integration	`features/pharmacy/PrescriptionQueue.test.tsx`	Pharmacist list polls at NEXT_PUBLIC_PRESCRIPTION_POLL_INTERVAL_MS, status tabs, oldest first, stops polling when tab hidden	FR-021, FR-025
FT-010	component	`features/pharmacy/DispenseDialog.test.tsx`	Quantities default to prescribed; note required below prescribed; all-zero blocked with hint to cancel; confirmation step; 409 shows current status and refreshes	FR-028

ID	Type	File	What it proves	Covers
FT-011	component	`features/queue/BookingForm.test.tsx`	Booking validation, queue number shown prominently after success, DUPLICATE_BOOKING confirmation re-submits with allowDuplicate and a new Idempotency-Key	FR-036, FR-037
FT-012	integration	`features/queue/QueueConsole.test.tsx`	Call next sends expectedLastCalledNumber from latest data; button disabled while pending; 409 QUEUE_STATE_CHANGED refetches and shows toast; complete/skip/requeue/cancel actions per status	FR-039, FR-042
FT-013	integration	`features/public-queue/PublicQueueDisplay.test.tsx`	Renders serving/waiting numbers only; polling with If-None-Match; stale-data banner after 3 failed polls; 'my number' position computed client-side	FR-044, FR-045, NFR-004
FT-014	a11y	`e2e/a11y.spec.ts`	Playwright + axe-core on every route in routes.yaml: zero serious/critical violations; keyboard-only completion of login, consultation and dispense flows	NFR-015
FT-015	integration	`features/auth/useIdleLogout.test.ts`	Logout after NEXT_PUBLIC_IDLE_TIMEOUT_MINUTES without activity; activity resets timer; TanStack cache cleared on logout	FR-003, FR-010, NFR-009
FT-016	unit	`lib/api/problem.test.ts`	Problem Details parsing; mapping of every ErrorCode to a user message; unknown code fallback; requestId shown in error panel	FR-020
FT-017	e2e	`e2e/workflows.spec.ts`	Playwright against docker compose stack: Workflows A-E end to end (existing patient, new patient, telephone booking, public display update, call next)	FR-021, FR-049, NFR-004, NFR-014, NFR-023, NFR-027, NFR-028
FT-018	e2e	`e2e/responsive.spec.ts`	Screens at 360, 768, 1280 and 1920 px: no horizontal scroll, navigation usable, public display readable at 1920×1080	NFR-016